

Zvýšenie bezpečnosti vybraných šifrovacích algoritmov v prístupových systémoch

Oleksandr Mykhailyshyn

Technická univerzita v Košiciach, Fakulta elektrotechniky a informatiky, Slovenská republika

Abstrakt – Práca sa zaoberá bezpečnostnými mechanizmami, ktoré sú relevantné pre moderné systémy kontroly prístupu využívajúce bezkontaktné identifikátory RFID/NFC. Teoretická časť nevystupuje iba ako všeobecný prehľad, ale priamo nadväzuje na vytvorený prototyp: čítačku založenú na ESP32 a RC522, serverovú aplikáciu v jazyku C++ a lokálne úložisko SQLite s auditnou stopou. Jadrom návrhu je otázka, ako zvýšiť dôveryhodnosť celého toku udalosti od načítania UID až po rozhodnutie o prístupe a neskoršie overenie auditu.

V texte preto rozoberám autentifikáciu hardvérových požiadaviek pomocou HMAC-SHA-256, ochranu proti opakovaniu správ cez monotónne `hw_seq`, vnútorný frame chránený mechanizmom AEAD XChaCha20-Poly1305 a auditný log navrhnutý tak, aby bolo možné odhaliť dodatočnú manipuláciu. Súčasne otvorene pomenúvam aj hranice prototypu: UID karty je v tomto riešení identifikátor, nie kryptografický dôkaz identity, databáza je uložená lokálne a základný experimentálny režim nevyžaduje povinné TLS. Tieto obmedzenia nie sú obchádzané, ale slúžia ako súčasť realistického hodnotenia toho, čo prototyp chráni dobre a kde zostáva priestor na ďalšie rozšírenie.

KLúčové slová – kontrola prístupu, RFID, NFC, kryptografia, AEAD, XChaCha20-Poly1305, HMAC, HKDF, anti-replay, audit log.

I. ÚVOD

Systémy kontroly prístupu (*Access Control Systems*) sú jedným z tých prvkov infraštruktúry, pri ktorých sa fyzická a informačná bezpečnosť stretávajú priamo v jednom bode. Na prvý pohľad ide iba o rozhodnutie, či sa dvere otvoria alebo nie. V skutočnosti sa za tým skrýva identifikácia používateľa, vyhodnotenie pravidiel, práca s tajomstvami, komunikácia po sieti a napokon aj audit, ktorý má po incidente vysvetliť, čo sa stalo.

Práve pri jednoduchších riešeniach býva rozdiel medzi funkčným a bezpečným systémom najvýraznejší. V praxi nie je problém zostaviť čítačku, ktorá po načítaní UID pošle hodnotu na server a čaká na odpoveď. Takýto návrh je však zraniteľný takmer okamžite: stačí zachytiť požiadavku, zopakovať ju v inom čase, podvrhnúť vlastný identifikátor alebo upraviť dáta v úložisku. Ak systém ukladá surové UID a pritom nemá dôveryhodný audit, po incidente zostáva iba veľmi slabá stopa.

Táto diplomová práca vychádza práve z takéhoto praktického pozorovania. Cieľom nebolo navrhnuť teoreticky maximálne silný systém bez ohľadu na cenu a zložitosť, ale postaviť prototyp, na ktorom sa dá ukázať, ktoré bezpečnostné mechanizmy majú v prístupových systémoch skutočný prínos. Výsledkom je kombinácia hardvéru (ESP32 + RC522), serverovej aplikácie v jazyku C++ a webového rozhrania, v ktorom sa spravujú čítačky, dvere, roly, karty aj auditné záznamy.

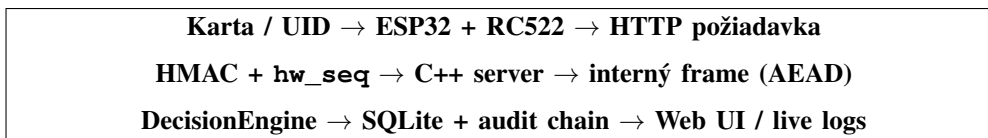
Bezpečnosť riešenia stojí na niekoľkých navzájom previazaných vrstvách:

- autentifikácia hardvérových požiadaviek pomocou HMAC-SHA-256[5],
- šifrovanie a integrita interného payloadu pomocou AEAD XChaCha20-Poly1305[1], [2], [3],
- ochrana proti replay útokom cez monotónne sekvencie a *ReplayWindow*,
- ochrana súkromia identifikátorov kariet pomocou HMAC hashovania s *pepper*,
- auditná stopa navrhnutá tak, aby bolo možné odhaliť dodatočnú manipuláciu pomocou HMAC hash chain a anchor bodov[9].

Dôležité je, že jednotlivé vrstvy v prototypu neplnia tú istú úlohu. Hardvérová čítačka neposiela na server priamo AEAD šifrovaný payload; odosiela HTTP požiadavku autentifikovanú pomocou

HMAC a monotónneho `hw_seq`. Až server po overení tejto požiadavky vytvára interný kryptograficky chránený frame, ktorý vstupuje do rozhodovacej logiky. Táto hranica dôvery je v texte pomenovaná zámerne, pretože ovplyvňuje aj interpretáciu výsledkov: niečo iné znamená ochrana proti podvrhnutiu HTTP požiadavky a niečo iné ochrana interného frame alebo auditnej stopy.

Teoretická časť preto nevystupuje ako izolovaný prehľad pojmov. Sleduje tie oblasti, ktoré sa v prototypu naozaj objavili: model hrozieb, autentifikáciu zariadenia, anti-replay mechanizmy, návrh bezpečného protokolu, prácu s kľúčmi a auditovateľnosť. Praktická a analytická časť na to nadväzuje experimentmi, v ktorých sa porovnáva jednoduchší základný variant so zabezpečeným riešením.



Obr. 1 Zjednodušená end-to-end schéma prototypu a hraníc dôvery medzi vrstvami.

II. MOTIVÁCIA, CIELE A PRÍNOS PRÁCE

A. Motivácia

Motivácia tejto práce nie je iba teoretická. Počas návrhu a skladania prototypu sa opakovane ukazovalo, že najjednoduchšia funkčná verzia systému by sa dala postaviť veľmi rýchlo: prečítať UID, poslať ho na server a podľa odpovede otvoriť dvere. Takýto návrh však rieši iba funkcionálnosť, nie dôveryhodnosť. Hneď ako sa do úvahy vezme bežná lokálna sieť, databáza s kartami a potreba spätne vysvetliť incident, začnú byť zaujímavé celkom iné otázky.

Reálne prostredie totiž zahŕňa:

- nepredvídateľnú sieť (Wi-Fi, zdieľané siete, kompromitované zariadenia),
- úložisko v podobe databázy, ktorú je možné kopírovať a analyzovať,
- insider scenáre (administrátor, ktorý môže zneužiť oprávnenia alebo zmeniť logy),
- vstavané zariadenia s obmedzenými zdrojmi, ktoré často nemajú plnú podporu pre komplexné bezpečnostné protokoly.

Ak systém nepoužíva kryptograficky zabezpečené logovanie, útočník môže po incidente vymazať stopy. Ak systém neobsahuje anti-replay, je možné prístup opakovať bez karty. Ak systém ukladá surové UID, únik databázy priamo ohrozuje používateľov.

B. Ciele práce

Hlavné ciele práce sú:

- 1) Navrhnuť bezpečnostné požiadavky a model hrozieb pre prototyp prístupového systému.
- 2) Zaviesť kryptografické mechanizmy na úrovni komunikácie a spracovania:
 - AEAD (XChaCha20-Poly1305) pre dôvernoscť a integritu dát[1], [2],
 - HMAC-SHA-256 pre autentifikáciu požiadaviek hardvéru[5],
 - HKDF pre deriváciu kľúčov a oddelenie účelov[4].
- 3) Implementovať anti-replay mechanizmus založený na monotónnom `hw_seq` (perzistentne uloženom na ESP32) a *ReplayWindow* na serveri.
- 4) Navrhnuť a implementovať tamper-evident audit log s možnosťou verifikácie a detekcie truncation.
- 5) Stanoviť analytické ciele hodnotenia: simulovať vybrané útoky a prevádzkové chyby, zbierať metriky (úspech/neúspech útoku, dôvody zamietnutia, latenciu spracovania a záťaž) a porovnať základnú (baseline) implementáciu so zabezpečenou verziou s formulovaním záverov.

C. Prínos

Prínos práce je možné čítať v dvoch rovinách. Prvá je technická: vznikol funkčný prototyp, na ktorom sa dá ukázať, že aj pomerne malý prístupový systém môže mať jasne oddelené bezpečnostné vrstvy. Druhá rovina je metodická: práca nevychádza len z tvrdenia, že zvolený mechanizmus je „bezpečnejší“, ale pripravuje priestor na jeho meranie a porovnanie s jednoduchšou verziou riešenia.

Konkrétne ide o:

- demonštrácia realistického návrhu bezpečnej komunikácie pre IoT/embedded zariadenia,
- dôsledné oddelenie autentifikácie hardvéru od administrátorských oprávnení (HW nepoužíva admin token),
- ochrana súkromia používateľov pomocou hashovania identifikátorov kariet,
- auditovateľnosť a detekcia manipulácie logov pomocou kryptografických reťazcov,
- modulárna architektúra C++ projektu (kryptografia, protokol, rozhodovanie, úložisko, UI).

D. Rozsah riešenia a vedomé obmedzenia

V tejto práci nepovažujem za poctivé tváriť sa, že prototyp rieši všetky problémy produkčného prístupového systému. Niektoré voľby boli urobené zámerne, pretože cieľom bolo overiť konkrétne bezpečnostné mechanizmy a nie simulovať kompletne enterprise nasadenie.

Použitá RFID karta pracuje v jednoduchom režime, takže UID vystupuje ako identifikátor, nie ako kryptografický dôkaz identity. Databázová vrstva je postavená na SQLite, čo je pre prototyp a experimenty praktické, ale neznamená to vysokú dostupnosť, centralizovanú správu tajomstiev ani pohodlné škálovanie. Komunikácia hardvéru so serverom je v základnom režime chránená autentifikáciou a integritou na aplikačnej vrstve (HMAC + hw_seq), zatiaľ čo dôvernosť transportu cez TLS zostáva rozšírením pre ďalšie nasadenie.

Tieto obmedzenia teda nie sú slabinou textu, ktorú by bolo potrebné zakryť. Naopak, vytvárajú presný rámec pre interpretáciu výsledkov. Ak sa v analytickej časti ukáže zlepšenie odolnosti voči podvrhnutiu, replay alebo manipulácii s auditom, bude zrejmé, že ide o prínos navrhnutých mechanizmov v rámci definovaného prototypu, nie o všeobecné tvrdenie o všetkých prístupových systémoch.

Tabuľka 1
Vedomé obmedzenia prototypu a ich význam pre interpretáciu výsledkov

Oblasť	Aktuálne riešenie	Dôsledok pre prácu
Identita karty	UID slúži ako identifikátor, nie ako kryptografický dôkaz	Práca sa zameriava najmä na bezpečnosť komunikácie, serverovej logiky a auditu, nie na bezpečné smart karty.
Úložisko	Lokálna SQLite databáza	Riešenie je vhodné pre prototyp a experimenty; vysoká dostupnosť a centralizovaná správa tajomstiev nie sú predmetom práce.
Transport	Povinné TLS nie je v základnom režime nasadené	Dôvernosť prenosu UID nie je hlavnou demonštrovanou vlastnosťou; dôraz je na autentifikáciu zariadenia, anti-replay a auditovateľnosť.
Správa tajomstiev	Tajomstvá sú spravované pragmaticky pre potreby prototypu	Výsledky treba interpretovať ako experimentálny návrh, nie ako plnohodnotné produkčné nasadenie.

E. Metodika hodnotenia a analytické ciele

Pri podobných prototypoch sa často končí pri ukážke, že systém „funguje“. To však pre tému zameranú na bezpečnosť nestačí. Preto je hodnotenie od začiatku postavené tak, aby bolo možné overiť, či navrhnuté mechanizmy skutočne menia správanie systému pri útoku alebo chybe.

V praktickej časti sa preto nebude sledovať iba to, či server vráti `allow` alebo `deny`. Dôležité bude aj to, prečo k rozhodnutiu došlo, aká je úspešnosť jednotlivých útokov, akú latenciu zavádzajú ochranné mechanizmy a ako sa riešenie správa pri opakovaných alebo chybných požiadavkách. Porovnanie základného a zabezpečeného variantu je zvolené práve preto, aby analytická časť prirodzene vyrástla z cieľov definovaných v teórii a nepôsobila ako neskôr doplnený experimentálny dodatok.

III. VŠEOBECNÉ BEZPEČNOSTNÉ POJMY A TERMINOLÓGIA

Táto kapitola predstavuje základné pojmy, ktoré sa používajú v ďalších častiach práce.

Tabuľka 2
Prepojenie analytických cieľov s meranými ukazovateľmi

Scenár	Mechanizmus	Pozorovaný ukazovateľ
Podvrhnutie HW požiadavky	HMAC-SHA-256	Miera zamietnutia, dôvod <code>bad_signature</code> .
Replay zachytenej požiadavky	<code>hw_seq</code> + <code>ReplayWindow</code>	Miera zamietnutia, dôvod <code>replay</code> alebo <code>too_old</code> .
Manipulácia interného frame	AEAD + AAD	Neúspešné overenie integrity a zamietnutie správy.
Zásah do auditnej stopy	HMAC hash chain + anchor	Úspešnosť detekcie pri verifikácii auditu.
Prevádzková zátáž	Validácia vstupu + rozhodovanie	Latencia spracovania a správanie systému pri opakovaných požiadavkách.

A. CIA triáda a rozšírené bezpečnostné ciele

CIA triáda (dôvernosť, integrita, dostupnosť) je základný rámec, od ktorého sa pri bezpečnostnom návrhu zvyčajne začína. V prístupovom systéme však sama osebe nestačí. Pri praktickej implementácii sa veľmi rýchlo ukáže, že rovnako podstatné je vedieť, kto správu vytvoril, či ju nemožno znovu použiť a či sa dá po incidente späť overiť história udalostí. Preto je vhodné k CIA doplniť ešte niekoľko cieľov:

- autentickosť (*authenticity*) – schopnosť overiť pôvod správy alebo akcie,
- auditovateľnosť (*auditability*) – schopnosť späť preukázať rozhodnutia a udalosti,
- odolnosť voči replay (*replay resistance*) – schopnosť odmietnuť opakovanie starej správy.

V prototypy sa tieto ciele prejavujú veľmi konkrétne:

- Dôvernosť: útočník nemá získať citlivé údaje (napr. identifikátory, tokeny, kľúče).
- Integrita: útočník nemá zmeniť požiadavku alebo audit záznam bez detekcie.
- Dostupnosť: systém musí zvládnuť prevádzku bez výpadkov; zároveň sa musí brániť proti DoS v rámci možností prototypu.

Tabuľka 3
Mapovanie bezpečnostných cieľov na mechanizmy použité v prototypy

Cieľ	Mechanizmus v prototypy	Príklad narušenia
Dôvernosť	AEAD frame, neukladanie súrového UID	Čítanie citlivých údajov z payloadu alebo databázy.
Integrita	HMAC požiadavky, AEAD tag, hash chain auditu	Zmena <code>uid</code> , <code>door_id</code> alebo auditnej udalosti bez detekcie.
Dostupnosť	Limity vstupu, fail-closed, validácia	Pád parsera alebo vyčerpanie zdrojov pri chybných vstupoch.
Autentickosť	HMAC-SHA-256 pre hardvér	Vydávanie sa za legitímnu čítačku.
Odolnosť voči replay	<code>hw_seq</code> + <code>ReplayWindow</code>	Opakované použitie starej platnej požiadavky.
Auditovateľnosť	Tamper-evident audit log + verify nástroj	Dodatočné popretie udalosti alebo tichá úprava histórie.

B. AAA model: Authentication, Authorization, Accounting

Model AAA je v literatúre známy a pomerne stručný, no v kontexte tejto práce je užitočný práve preto, že sa dá priamo premietnuť do implementácie. Namiesto abstraktného delenia sa dá pri prototypy ukázať, kde sa ktorá časť naozaj nachádza:

- Authentication (autentifikácia) – overenie identity alebo pôvodu.
- Authorization (autorizácia) – rozhodnutie, či identita môže vykonať akciu.
- Accounting (účtovanie/audit) – záznam akcií a ich výsledkov.

V tomto prototypy je autentifikácia rozdelená na dve vrstvy. Karta slúži predovšetkým ako identifikátor používateľa, zatiaľ čo samotné zariadenie sa voči serveru preukazuje podpisom HMAC. Autorizácia prebieha v *DecisionEngine* na základe rolí a väzieb medzi čítačkou a dverami. Posledná časť modelu, teda accounting, nie je iba pomocný log pre administrátora; je to auditná

stopa, z ktorej má byť možné spätne overiť, čo sa stalo a či s históriou nebolo manipulované. V tomto bode sa teoretický model AAA prirodzene prepája s odporúčaniami pre digitálnu identitu a správu logov.[12], [13]

C. Riziko, hrozba, zraniteľnosť

V ďalších kapitolách sa opakovane pracuje s pojmami hrozba, zraniteľnosť a riziko, preto má zmysel ich hneď na začiatku oddeliť. Bez tohto rozlíšenia by bolo ťažké vysvetliť, prečo sa niektoré mechanizmy zavádzajú priamo do protokolu a iné iba ako odporúčanie pre budúce nasadenie.

Bezpečnostná analýza preto rozlišuje:

- hrozbu (threat) – potenciálnu príčinu incidentu (napr. útočník schopný zachytiť sieťovú komunikáciu),
- zraniteľnosť (vulnerability) – slabinu systému (napr. absencia anti-replay),
- riziko (risk) – kombináciu pravdepodobnosti a dopadu.

V tejto práci nie je cieľom vytvoriť úplný korporátny register rizík. Dôležitejšie je ukázať, ktoré hrozby sú pre zvolený prototyp realistické a ktoré mechanizmy ich vedia znížiť na prijateľnú mieru.

D. Modelovanie hrozieb

Pri návrhu prototypu sa ukázalo, že hrozby sa dajú najlepšie udržať pod kontrolou vtedy, keď sú pomenované ešte pred implementáciou. Ako orientačný rámec je preto vhodný model STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege). Nie všetky jeho časti sú v tejto práci rovnako dôležité; najviac sa týkajú tých vrstiev, ktoré prototyp naozaj obsahuje:

- Spoofing: podvrhnutie požiadavky (riešené HMAC),
- Tampering: zmena dát alebo logov (riešené AEAD, audit hash chain),
- Repudiation: popieranie udalostí (riešené auditom),
- Information disclosure: únik identifikátorov (riešené hashovaním UID v DB a odporúčaním TLS),
- DoS: zahltenie endpointu (čiastočne riešené limitmi a validáciou).

E. Autentifikačné faktory a pojem identity

V prístupových systémoch sa často hovorí o tom, že používateľ sa „identifikuje kartou“. Je dôležité rozlišovať dva pojmy: identifikácia (tvrdenie „kto som“) a autentifikácia (dôkaz, že tvrdenie je pravdivé). V praxi sa autentifikácia realizuje pomocou *faktorov*:

- niečo, čo mám (karta, token, zariadenie),
- niečo, čo viem (PIN, heslo),
- niečo, čím som (biometria).

RFID/NFC karta v jednoduchom režime (iba UID) je teda skôr identifikátor než plnohodnotný autentifikátor. V produkčných systémoch sa tento problém zvykne riešiť ďalším faktorom, napríklad PIN-om, alebo kartou s vlastným kryptografickým protokolom. V tejto práci sa tento rozdiel neskrýva; naopak, je súčasťou interpretácie výsledkov. Riziko sa kompenzuje inde – najmä autentifikáciou zariadenia voči serveru, anti-replay mechanizmami a auditovateľnosťou. Z pohľadu odporúčaní pre digitálnu identitu ide teda o prototyp s jednoduchším poverením používateľa, ale so silnejšou ochranou komunikačnej a serverovej vrstvy.[12], [19]

F. Zodpovednosť, nepopierateľnosť a forenzná pripravenosť

V praxi nestačí iba rozhodnúť *ALLOW/DENY*. Organizácia potrebuje vedieť *kedy, kde a prečo* k rozhodnutiu došlo. Tento aspekt sa často označuje ako accountability (zodpovednosť) a je úzko spojený s auditovateľnosťou.

Pojem *nepopierateľnosť* (non-repudiation) sa v kryptografii spája najmä s digitálnymi podpismi, ale v kontexte prístupových systémov má aj praktický význam: systém by mal vedieť preukázať konzistentný reťaz udalostí a minimalizovať možnosti spätnej manipulácie. To je motivácia pre tamper-evident logovanie a pre dôsledné zaznamenávanie kontextu (identifikátor čítačky, dverí, čas, výsledok, dôvod zamietnutia).

G. Bezpečnostné požiadavky ako merateľné tvrdenia

Pri návrhu bezpečnostného riešenia je užitočné formulovať požiadavky ako overiteľné tvrdenia. Napríklad:

- *Systém odmietne opakovanie tej istej požiadavky aj v prípade, že bola zachytená na sieti.*
- *Únik databázy nesmie priamo odhaliť surové identifikátory kariet.*
- *Po manipulácii s auditným záznamom je možné určiť prvé miesto nekonzistencie.*

Takto formulované požiadavky sa dajú priamo mapovať na mechanizmy (HMAC, anti-replay, HMAC hashovanie UID, hash chain) a v ďalších častiach práce poskytujú jasné kritériá, čo sa považuje za „zvyšenie bezpečnosti“.

IV. SYSTÉMY KONTROLY PRÍSTUPU A PRÍSTUPOVÉ MODEL Y

A. Fyzická vs. logická kontrola prístupu

Fyzická kontrola prístupu rieši vstupy do budov a zón. Logická kontrola prístupu rieši prístup k dátam a službám. V oboch prípadoch sa používa rovnaký princíp: subjekt (používateľ) vykoná akciu, systém overí identitu a rozhodne podľa politiky.[17]

B. Modely kontroly prístupu

Z teórie sú známe rôzne modely:

- DAC (Discretionary Access Control) – vlastník objektu určuje prístup.
- MAC (Mandatory Access Control) – prístup riadi centrálna politika (napr. bezpečnostné úrovne).
- RBAC (Role-Based Access Control) – oprávnenia sú priradené rolám, používatelia majú roly.
- ABAC (Attribute-Based Access Control) – rozhodovanie podľa atribútov subjektu, objektu a kontextu.

V prototypovej implementácii je použitý RBAC prístup: karty majú roly (napr. *admin*), dvere majú povolené roly a existuje väzba medzi *reader_id* a *door_id*. Tento model je dostatočne jednoduchý pre demo, ale zároveň realistický a dobre nadväzuje na klasické RBAC modely popísané v literatúre.[17], [18]

C. Bezpečnostné požiadavky na API a server

Keďže prototyp obsahuje webové API (administrátorské aj hardvérové), je vhodné reflektovať typické API riziká, napríklad podľa OWASP API Security Top 10 a širšieho rámca OWASP ASVS.[8], [16] V kontexte prístupového systému sú kritické najmä:

- Broken Authentication: slabé overenie identity klienta,
- Broken Object Level Authorization: nedostatočné overenie oprávnení pri práci s objektmi (dvere, čítačky, karty),
- Security Misconfiguration: debug endpointy alebo slabé nastavenia.

Praktická implementácia preto oddelí uje admin token (iba pre UI) od HW autentifikácie (HMAC).

D. Zóny, kontext a väzby medzi komponentmi

Vo fyzickej kontrole prístupu sa často pracuje so zónami (napr. verejná zóna, kancelárska zóna, technická zóna). Politika prístupu potom vyjadruje, ktoré roly alebo skupiny používateľov môžu vstupovať do konkrétnych zón a v akom čase. Aj keď prototyp používa zjednodušený model *dvere* → *povolené roly*, ide o rovnaký princíp: dvere reprezentujú hranicu zóny.

Dôležitým bezpečnostným prvkom je väzba (binding) medzi čítačkou a dverami. Ak by útočník vedel poslať na server udalosť s ľubovoľným *door_id*, mohol by zneužiť legitímnu čítačku na otvorenie iných dverí. Preto sa v prístupových systémoch používa:

- fyzické opatrenie (umiestnenie čítačky pri konkrétnych dverách),
- konfiguračná väzba (server pozná, ku ktorým dverám čítačka patrí),
- a v ideálnom prípade aj kryptografická identita zariadenia.

V prototypovej architektúre sa väzba realizuje databázovým pravidlom *reader_id* ↔ *door_id*. Všeobecne ide o ukážku toho, ako sa *kontext* stáva súčasťou autorizácie.

E. Princíp najmenej oprávnení a oddelenie právomocí

Princíp *least privilege* hovorí, že každá komponenta má mať len také oprávnenia, ktoré nevyhnutne potrebuje. V prístupovom systéme to znamená, že:

- hardvérové zariadenie nemá mať administrátorské oprávnenia,
- administrátorské rozhranie a jeho tokeny majú byť oddelené od mechanizmov zariadenia,
- a databázové operácie majú byť obmedzené na minimum potrebné pre rozhodovanie a audit.

Oddelenie právomocí znižuje dopad kompromitácie jednej časti systému. Ak by napríklad útočník získal kontrolu nad čítačkou, stále by nemal automaticky získať možnosť meniť prístupové pravidlá alebo exportovať databázu.

V. BEZPEČNOSTNÉ HROZBY A ŠPECIFIKÁ RFID/NFC TECHNOLOGIÍ

RFID a NFC pôsobia na prvý pohľad jednoducho: karta sa priloží, čítačka prečíta identifikátor a systém rozhodne. Práve táto jednoduchosť však zvädza k podceneniu hrozieb. V praxi nebýva problémom iba samotná karta, ale celý reťazec od rádiovkej komunikácie až po serverové spracovanie. NIST SP 800-98 preto nepozera na RFID ako na izolovaný prvok, ale ako na súčasť väčšieho bezpečnostného systému[7].

Pre túto prácu je dôležité jedno rozlíšenie. Prototyp nepoužíva kryptograficky silnú kartu s mutual authentication; pracuje s UID ako s identifikátorom, nad ktorým sa až následne uplatnia serverové bezpečnostné mechanizmy. To je vedomé rozhodnutie a treba ho vnímať už pri čítaní celej kapitoly.

A. Typické útoky na RFID/NFC

Pri RFID/NFC sa často spomínajú tie isté triedy útokov, ale nie všetky majú v každom projekte rovnakú váhu. V našom prototypu sú najdôležitejšie najmä tie, ktoré priamo súvisia s tým, že karta neposkytuje kryptografický dôkaz identity.

- **Skimming a eavesdropping:** útočník sa snaží zachytiť komunikáciu medzi čítačkou a kartou. Krátky dosah NFC pomáha, ale s lepšou anténou alebo v rušnom prostredí sa nedá rátať s tým, že odpočúvanie je „prakticky nemožné“.
- **Cloning a emulácia:** ak systém stojí iba na UID, z klonovania sa stáva veľmi realistický problém. Útočník nepotrebuje poznať prístupovú politiku; stačí mu vedieť napodobniť identifikátor.
- **Relay útoky:** tu nejde o kopírovanie karty, ale o predĺženie jej dosahu. Čítačka tak komunikuje s legitímnou kartou, ktorá je fyzicky inde.
- **Manipulácia čítačky:** ak útočník nahradí čítačku vlastným zariadením alebo sa pripojí na jej výstup, problém sa presúva z rádiovkej vrstvy na aplikačnú a sieťovú vrstvu.

B. Mitigácie a odporúčané protiopatrenia

Nie je realistické tvrdiť, že jeden mechanizmus vyrieši celý problém RFID/NFC. Obrana sa preto skladá z viacerých vrstiev:

- **Kryptografická karta:** ak karta vie preukázať znalosť tajomstva, znižuje sa význam samotného UID.
- **Druhý faktor:** PIN alebo biometria dávajú zmysel najmä tam, kde by jediné zneužitie karty malo vysoký dopad.
- **Obrana proti relay:** distance bounding a prísnejšie časové limity sú technicky zaujímavé, ale v jednoduchom prototypu by zbytočne zvyšovali zložitosť.
- **Organizačné opatrenia:** fyzická kontrola vstupov, kamera alebo revízia oprávnení sú menej elegantné než kryptografia, no v reálnej prevádzke bývajú veľmi účinné.

V tejto práci sa preto nehrá na to, že samotná karta je „silný autentifikátor“. Posilňuje sa až vyššia vrstva: čítačka sa musí serveru preukázať pomocou HMAC, požiadavka má monotónne hw_seq a výsledok spracovania sa zapisuje do auditu tak, aby ho bolo možné dodatočne overiť.

C. Príklad známych slabín: MIFARE Classic

Historicky boli široko nasadené karty typu MIFARE Classic, ktoré používajú proprietárny algoritmus CRYPTO1. Výskum ukázal vážne slabiny v tomto mechanizme a umožnil praktické útoky vedúce k získaniu kľúčov a klonovaniu kariet. Z toho vyplýva dôležitá poučka: spoliehanie sa na utajenie algoritmu alebo na UID ako jediný faktor je nedostatočné. Aj keď konkrétny prototyp v práci nepoužíva autentifikáciu na úrovni karty, návrh kompenzuje riziká na úrovni servera a protokolu (HMAC, anti-replay, audit).

Tabuľka 4
Vybrané hrozby a ich väzba na protiopatrenia v prototypu

Hrozba	Protiopatrenie	Poznámka
Spoofing čítačky	HMAC podpis požiadavky	Chráni pôvod a integritu HTTP požiadavky.
Replay požiadavky	Monotónne <code>hw_seq</code> + Replay-Window	Účinné aj po reštarte zariadenia pri zachovaní perzistentného stavu.
Manipulácia interných dát	AEAD XChaCha20-Poly1305 + AAD	Chráni payload a viaže hlavíčku na šifrované dáta.
Únik databázy	HMAC(<code>card_id</code>) s pepper	Obmedzuje priamu použiteľnosť uniknutých identifikátorov.
Mazanie alebo úprava logov	HMAC hash chain + anchor	Umožňuje neskoršiu detekciu nekonzistencie auditu.

D. Bezpečnostné dôsledky pre návrh prototypu

Z predchádzajúcich bodov pre prototyp vyplývajú štyri jednoduché pravidlá. Po prvé, UID sa používa len ako identifikátor vstupu do politiky. Po druhé, dôveru prenášame z karty na čítačku a serverové spracovanie. Po tretie, každá požiadavka musí niesť stopu čerstvosti, inak by ju bolo možné opakovať. A napokon, ani databáza nesmie obsahovať viac údajov, než je skutočne potrebné. Tento rámec sa potom priamo odráža v praktickej implementácii.

E. Hrozby na komunikačnej vrstve a v sieťovej infraštruktúre

Aj keď je fyzická technológia (RFID/NFC) často v centre pozornosti, v moderných riešeniach je rovnako kritická sieťová komunikácia medzi čítačkou a serverom. Medzi typické hrozby patria:

- Man-in-the-Middle (MITM): útočník zmení alebo presmeruje komunikáciu (napr. kompromitovaný prístupový bod Wi-Fi).
- Replay útoky: zachytená platná požiadavka sa opakuje s cieľom dosiahnuť rovnaký výsledok.
- Podvrhnutie klienta: útočník sa vydáva za čítačku a posiela vlastné požiadavky.
- Injekcia a deserializácia: nevalidovaný vstup môže viesť k pádom, obchádzaniu logiky alebo k zneužitiu parsera.

Práve tieto hrozby sú motiváciou pre použitie HMAC na autentifikáciu zariadenia a pre dôsledné limity a validáciu vstupov na serveri.

F. Hrozby na serverovej strane a v API

Server spracúva rozhodnutia a drží konfiguráciu (dvere, roly, väzby). To z neho robí najcennejší cieľ. Okrem klasických sieťových útokov (DoS) sú relevantné najmä:

- nesprávna autorizácia: chyby v kontrole oprávnení pri operáciách typu *CRUD*,
- únik tajomstiev: logovanie kľúčov, nesprávne nastavené prístupové práva, nechcené debug endpointy,
- privilege escalation: získanie administrátorských práv cez slabú autentifikáciu alebo zraniteľnosť v UI,
- kompromitácia závislostí: použitie knižníc s bezpečnostnými chybami.

Z teoretického hľadiska je dôležité uplatniť *defense-in-depth*: aj keď jedna vrstva zlyhá, ďalšie vrstvy majú minimalizovať dopad.

G. Útoky po incidente: manipulácia auditnej stopy

Po úspešnom preniknutí do systému sa útočník často snaží znížiť šancu na odhalenie. Najčastejšie to znamená upraviť logy alebo ich čiastočne odstrániť. Preto je auditovateľnosť a integrita logov dôležitou súčasťou bezpečnostného návrhu, nie iba „doplňkom“. Mechanizmy ako hash chain a anchor umožňujú neskoršie overenie, či auditná stopa bola zachovaná v konzistentnom stave.

VI. KRYPTOGRAFICKÉ ZÁKLADY RELEVANTNÉ PRE PRÍSTUPOVÉ SYSTÉMY

V tejto časti nejde o „prehľad všetkej kryptografie“. Potrebné je skôr pochopiť, čo z použitých mechanizmov v prototypu skutočne robí a čo od nich naopak očakávať nemožno. Práve zámena týchto rolí býva častým zdrojom chýb: hash sa zamieňa za autentifikáciu, TLS za autorizáciu a šifrovanie za celkovú bezpečnosť systému.

A. Hashovacie funkcie a integrita

Hashovacia funkcia je v bezpečnostnom návrhu užitočná, ale jej možnosti sú obmedzené. Vie vytvoriť pevne dlhý odtlačok vstupu a dobre odhalí náhodnú alebo úmyselnú zmenu dát. Nevyrieši však otázku pôvodu správy. Ak útočník zmení obsah, nič mu nebráni prepočítať aj nový hash. Preto samotný hash nestačí všade tam, kde chceme vedieť nielen to, že sa správa zmenila, ale aj to, kto ju vytvoril.

B. MAC a HMAC

MAC je mechanizmus, ktorý s využitím tajného kľúča zabezpečuje integritu a autentickosť správy. HMAC je konkrétna konštrukcia MAC nad hashovacou funkciou a je štandardizovaná vo FIPS 198-1; pôvodná všeobecná konštrukcia HMAC je popísaná aj v RFC 2104.[5], [10] HMAC sa používa v prototypovej implementácii pre autentifikáciu požiadaviek hardvéru.

Dôležité implementačné poznámky:

- Kanonizácia vstupu: HMAC sa musí počítat nad jednoznačnou reprezentáciou správy. Ako príklad jednoznačnej kanonizácie sa môže použiť reťazec "POST /api/hw/uid\n"+ raw JSON body. Táto voľba zabezpečuje, že aj zmena whitespace alebo poradie polí (ak by JSON serializácia nebola stabilná) môže ovplyvniť podpis. Prakticky preto zariadenie posielajúce *raw* telo a server podpisuje rovnaký reťazec.
- Konštantnočasové porovnanie: výsledný HMAC sa musí porovnávať konštantným časom, aby sa minimalizovalo riziko timing útokov.
- Oddelenie kľúčov: kľúč pre HW autentifikáciu nesmie byť zdieľaný s inými účelmi (napr. audit). HKDF umožňuje oddeliť podkľúče[4].

C. Výber parametrov a bezpečnostná úroveň

Pri použití kryptografických primitív je vhodné uvažovať o bezpečnostnej úrovni (napr. 128-bitová bezpečnosť) a o tom, aké parametre to implikuje. V praxi to znamená:

- používať dostatočne dlhé kľúče (typicky 256 bitov pre symetrické primitíva),
- nepoužívať zastarané hashovacie funkcie (napr. MD5, SHA-1) pre bezpečnostné účely,
- a vyhnúť sa „zvláštnym“ vlastným formátom, ktoré môžu skryť chyby v spracovaní.

V prístupových systémoch je často kľúčová dlhodobá udržateľnosť: zariadenia môžu byť nasadené roky. Preto je vhodné mať v protokole priestor pre kryptografickú agilitu (verzovanie kľúčov, možnosť rotácie) a vyhnúť sa algoritmom, ktoré sú viazané na špecifický hardvér.

D. Bezpečná práca s pamäťou a citlivými údajmi

Kryptografické kľúče a medzi-výsledky (napr. HMAC tag, plaintext pred šifrovaním) predstavujú citlivé údaje aj v pamäti procesu. Z teoretického hľadiska sa odporúča:

- minimalizovať dobu, počas ktorej sú citlivé údaje v pamäti,
- prepisovať buffre po použití, ak knižnica poskytuje bezpečné API,
- a vyhýbať sa kopírovaniu citlivých údajov do logov alebo výnimiek.

Tieto zásady sú obzvlášť relevantné na serverovej strane, kde proces môže bežať dlhodobo a kde sa môžu objaviť *core dumps* alebo diagnostické výpisy.

E. Symetrické šifrovanie a AEAD

Symetrické šifrovanie je vhodné pre embedded zariadenia aj servery kvôli výkonu. Samotné šifrovanie však nerieši integritu. V praxi sa preto používa AEAD, ktoré spája dôvernosť a integritu do jednej operácie.

1) *ChaCha20 a Poly1305*: ChaCha20 je stream cipher navrhnutý tak, aby mal vysoký výkon v softvéri. Poly1305 je univerzálny autentifikátor (MAC) používaný v AEAD režime ChaCha20-Poly1305. Ich kombinácia je štandardizovaná v RFC 8439 a zapadá do širšieho modelu AEAD rozhraní popísaného v RFC 5116.

citeRFC8439,RFC5116

2) *XChaCha20-Poly1305 a nonce manažment*: AEAD konštrukcie sú „krehké“ voči zneužitiu nonce: opakovanie nonce pod rovnakým kľúčom môže viesť k vážnym únikom informácií. XChaCha20-Poly1305 rozširuje nonce na 192 bitov, čo umožňuje bezpečnejšie používanie náhodných nonce pre dlhodobu žijúce kľúče

citeXChaChaDraft,LibsodiumXChaChaDoc. V prototypovej implementácii nonce generuje server.

3) *Associated Data (AAD)*: AEAD podporuje *Associated Authenticated Data (AAD)*: dáta, ktoré sa nešifrujú, ale ich integrita je kryptograficky chránená. V prístupovom protokole je AAD použité pre hlavičku frame. Tým sa zabezpečí, že zmena `reader_id`, `door_id` alebo `seq` spôsobí zlyhanie overenia tagu.

F. Náhodnosť, nonce a praktické generovanie

Mnohé bezpečnostné protokoly stoja na predpoklade, že niektoré hodnoty sú nepredvídateľné (napr. nonce, kľúče). V embedded prostredí však môže byť generovanie kvalitnej náhodnosti problematické: zariadenie nemusí mať dostatok entropie a môže sa spoliehať na deterministické zdroje. Preto je v praxi vhodné:

- používať knižnice, ktoré riešia inicializáciu RNG a poskytujú správne API,
- generovať nonce na serveri, ak zariadenie nevie garantovať kvalitu náhodnosti,
- a zabezpečiť, aby sa nonce pod rovnakým kľúčom neopakovala.

Rozšírený nonce pri XChaCha20-Poly1305 znižuje riziko kolízie aj pri náhodných noncoch, čo je výhodné v systémoch s dlhšou životnosťou kľúčov.[3]

G. Domain separation a kontext v derivácii kľúčov

Aj keď je HKDF formálne jednoduchý, v praxi je kľúčové správne nastaviť *context/label* (parameter `info`). Domain separation znamená, že z rovnakého master secret sa odvodí rôzne podkľúče tak, aby sa nedali zameniť. Typickým prístupom je použiť popis účelu, napríklad `"hw-auth"`, `"aad-frame"` alebo `"audit-chain"`. Takéto označovanie je jednoduché, ale výrazne znižuje riziko, že sa jeden kľúč „nechtiac“ použije inde.

H. Kryptografia vs. bezpečnosť implementácie

Bezpečnosť algoritmov môže byť oslabená implementáciou. Typické chyby sú porovnávanie MAC ne-konštantným časom, únik citlivých údajov do logov, neobmedzené spracovanie veľkých vstupov (vyčerpanie zdrojov) a nedostatočné ošetrovanie chýb. Preto sa v bezpečnom návrhu uplatňuje zásada: primitíva sú nutné, ale nestačia. Rovnako dôležité je robustné spracovanie vstupov, limity a jednotné chybové správanie.

VII. ZÁSADY NÁVRHU BEZPEČNÝCH PROTOKOLOV

Pri čítaní bezpečnostnej literatúry je ľahké nadobudnúť dojem, že rozhoduje najmä výber algoritmu. Pri praktickom protokole to tak nebýva. O výsledku často rozhodne obyčajná vec: ne-jednoznačný formát, nevhodná chybová hláška alebo príliš dôverčivý parser. Preto má táto kapitola možno menej „kryptografický“ charakter, ale pre výslednú bezpečnosť je rovnako dôležitá.

A. Fail-closed správanie

Pri prístupovom systéme býva prirodzené pýtať sa, ako zabezpečiť, aby sa dvere otvorili vždy, keď majú. Bezpečnostný návrh si však musí rovnako tvrdo položiť opačnú otázku: čo sa stane, keď si systém nie je istý? V tejto práci je odpoveď jednoznačná. Ak zlyhá HMAC, validácia vstupu, anti-replay alebo interná konzistencia dát, výsledkom nie je „skúsme to ešte nejak zachrániť“, ale zamietnutie. Takýto prístup je menej pohodlný pri ladení, no bezpečnejší v prevádzke.

B. Jednoznačná serializácia a kanonizácia

Pri podpisovaní alebo šifrovaní je kritické, aby všetky strany pracovali s rovnakou reprezentáciou dát. Ak by napríklad hardvér podpisoval JSON s iným poradím polí než server, mohlo by dôjsť k chybnému odmietnutiu. Naopak, ak by server akceptoval viacero reprezentácií, útočník môže hľadať *gadgets* na bypass podpisu. Preto prototyp podpisuje *raw body* a server používa rovnaký presný vstup do HMAC.

C. Verzovanie formátu a algoritmov

Bezpečné systémy musia počítať s evolúciou: zmeny formátu, rotácia kľúčov, prípadne migrácia algoritmov. Preto protokol obsahuje polia ako `key_version` a interné limity (maximálne dĺžky polí, verzovanie), čo znižuje riziko „tichých“ nekompatibilit.

D. Bezpečné spracovanie chýb a jednotné odpovede

Chybové správy sú často podceňovaný komunikačný kanál. Ak systém vracia detailné dôvody zlyhania (napr. „nesprávny podpis“, „neznáma čítačka“, „príliš staré seq“), útočník môže tieto informácie využiť na *enumeráciu* a postupné zlepšovanie útoku. Zároveň však prevádzka potrebuje vedieť, čo sa deje. Rozumný kompromis je: jednotná odpoveď smerom k nedôveryhodnému klientovi a detailný dôvod iba v internom logu a administrácii.

E. Ochrana proti DoS a vyčerpaniu zdrojov

DoS (Denial of Service) je relevantný aj pre prístupové scenáre, pretože môže spôsobiť výpadok vstupov alebo núdzový režim. Na úrovni protokolu a servera je vhodné uplatniť limity veľkosti požiadaviek, rýchlu validáciu pred náročnými operáciami (napr. pred dešifrovaním), a jednoduchú ochranu proti floodu (rate limiting). Takéto opatrenia nezrušia DoS úplne, ale zvyšujú odolnosť.

F. Bezpečnosť knižníc a závislostí

Kryptografické primitíva je vhodné realizovať cez overené knižnice. Z teoretického hľadiska to znamená minimalizovať „vlastnú kryptografiu“, sledovať aktualizácie a mať testy, ktoré zachytia regresie. V prototypovej implementácii je kryptografická vrstva postavená na knižnici `libsodium`. [3]

G. Mapovanie bezpečnostných cieľov na mechanizmy

Tabuľka 5 sumarizuje, ktorý mechanizmus v prototypovom systéme poskytuje akú bezpečnostnú vlastnosť.

Bezpečnostná vlastnosť	Mechanizmus v prototypu
Autenticnosť požiadavky od HW	HMAC-SHA-256 nad HTTP požiadavkou
Integrita metadát (reader/door/seq)	AEAD AAD väzba hlavičky frame
Dôvernosť interného payloadu	AEAD XChaCha20-Poly1305
Ochrana proti replay	hw_seq + ReplayWindow (sliding window)
Súkromie identifikátora v DB	HMAC(card_id) s pepper
Detekcia manipulácie auditu	HMAC hash chain + audit anchor

Tabuľka 5
Mapovanie bezpečnostných cieľov na mechanizmy v prototypovom systéme

VIII. SPRÁVA KRYPTOGRAFICKÝCH KLÚČOV

V prototypoch sa o klúčoch často píše iba jednou vetou: „uložia sa do konfigurácie“. To síce môže byť pravda, ale z bezpečnostného pohľadu to nič nevysvetľuje. Aj jednoduchý systém potrebuje vedieť, odkiaľ sa klúče berú, na čo presne slúžia, kedy sa menia a čo sa stane po incidente. NIST SP 800-57 tento problém opisuje ako životný cyklus klúča [6]; v praxi je to skôr séria nepríjemných, ale nevyhnutných prevádzkových otázok.

A. Oddelenie klúčov podľa účelu

Dobrá prax je používať odlišné klúče pre odlišné účely:

- klúč pre HW autentifikáciu (HMAC podpis HTTP),
- klúč pre AEAD šifrovanie payloadu,
- klúč pre audit hash chain,
- *pepper* pre hashovanie identifikátorov kariet.

Použitie jedného klúča pre viac účelov môže viesť k *cross-protocol* útokom alebo k nejasnostiam pri rotácii. Preto je vhodné odvodiť podklúče z hlavného tajomstva pomocou HKDF [4].

B. Verzovanie a rotácia klúčov

V prototypovej implementácii má čítačka priradenú `key_version`. Pri rotácii sa v databáze nastaví nová verzia a server môže dočasne akceptovať aj predchádzajúcu verziu (*allow-previous*), aby sa minimalizoval výpadok pri prechode.

C. Ukladanie kľúčov a bezpečnosť servera

Kľúče sú v prototypovej verzii uložené v konfiguračnom súbore servera (ako hex reťazce) a načítané pri štarte. Ide o pragmatické riešenie vhodné pre experimentálny prototyp, no z pohľadu produkčnej bezpečnosti predstavuje vedomý kompromis. V produkčnom nasadení by bolo vhodné použiť dedikované úložisko tajomstiev (napr. HSM, Vault), obmedziť prístupové práva procesu a oddeliť správu tajomstiev od aplikačného servera.

D. Provisioning a registrácia zariadení

V produkčnom systéme je dôležitá fáza provisioningu: ako sa čítačka prvýkrát dostane do systému a ako sa jej priradia tajomstvá. Teoreticky existujú dva prístupy: predzdieľané tajomstvo (PSK) vložené pri výrobe/inštalácii alebo bootstrap protokol (napr. na báze certifikátov). PSK je jednoduchšie, ale horšie škáluje a vyžaduje disciplinované nakladanie s tajomstvom počas celej životnosti zariadenia.

E. Kompromitácia kľúča a postup obnovy

Aj pri dobrej implementácii treba počítať s kompromitáciou zariadenia alebo únikom kľúča. Incident plán by mal riešiť, ako rýchlo vieme zablokovať kompromitované zariadenie, ako prebehnú rotácia kľúčov a ako identifikujeme udalosti, ktoré mohli byť ovplyvnené kompromitáciou.

F. Evidencia verzií a migračné stratégie

Migrácia bezpečnostných mechanizmov je citlivá, pretože výpadok znamená praktický problém v prevádzke. Z teoretického hľadiska sa často používa fázovaný rollout, krátke obdobie dual-acceptance (prijíma sa stará aj nová verzia) a následná revokácia starších verzií.

Pre digitálne identity a autentifikátory sú relevantné aj odporúčania NIST SP 800-63-4, ktoré zdôrazňujú riadenie autentifikátorov, životný cyklus a procesy obnovy.

IX. NÁVRH BEZPEČNÉHO KOMUNIKAČNÉHO PROTOKOLU

A. End-to-end tok udalosti

V tejto práci sa z bezpečnostného hľadiska oplatí sledovať udalosť od úplného začiatku. Čítačka prečíta UID, zariadenie zvýši `hw_seq`, vytvorí krátku HTTP požiadavku a podpíše ju. Server po prijatí ešte stále „nepovoľuje vstup“; najskôr overí pôvod a základnú konzistenciu správy. Až potom sa z požiadavky vytvorí interný secure frame, ktorý vstupuje do rozhodovacej logiky.

Prakticky teda tok vyzerá takto:

- 1) RC522 načíta UID karty.
- 2) ESP32 inkrementuje perzistentné `hw_seq`.
- 3) Do `/api/hw/uid` sa odošle JSON s `uid`, `reader_id`, `door_id` a `hw_seq`.
- 4) Hlavička `X-HW-Signature` nesie HMAC nad presným telom požiadavky.
- 5) Server overí podpis a validuje vstup.
- 6) Zo správy vytvorí interný frame, kde sa `hw_seq` použije ako `seq`.
- 7) *DecisionEngine* vyhodnotí pravidlá a výsledok sa uloží do auditu.
- 8) *EventBus* sprístupní runtime udalosti pre administráciu.

Takéto rozdelenie vrstiev je dôležité aj pri interpretácii výsledkov. HMAC medzi zariadením a serverom rieši autenticitu a integritu požiadavky. AEAD frame na serverovej strane potom rieši vnútorné spracovanie, väzbu metadát a auditovateľnosť ďalších krokov.

B. Formát správ, validácia vstupu a obrana pred parser útokmi

V prístupovom systéme predstavuje vstupná správa (napr. JSON požiadavka) hranicu medzi nedôveryhodným svetom a internou logikou servera. Aj keď je správa podpísaná, server musí počítať s tým, že útočník sa môže pokúsiť o:

- poslať extrémne veľké alebo špeciálne formátované vstupy (vyčerpanie pamäte),
- využiť rozdiely v parsovaní (napr. rozdiel medzi „číslo“ a „reťazec“),
- obísť kontrolu pomocou duplicitných polí alebo neštandardných kódovaní.

Z teoretického hľadiska sa odporúča spracovať vstup v dvoch krokoch:

- 1) syntaktická validácia (správny JSON, maximálna veľkosť, povolené znaky),
- 2) semantická validácia (povinné polia, rozsahy hodnôt, typy, konzistentnosť).

Až po úspešnej validácii má zmysel vykonávať ďalšie operácie (napr. dešifrovanie, databázové dotazy). Tento postup znižuje riziko DoS a zlepšuje robustnosť.

C. Hranice dôvery a spracovanie metadát

Nie všetky polia v správe majú rovnaký význam. Napríklad `uid` je identifikátor karty, no nie je dôkazom autenticity. Naopak `reader_id` a `door_id` sú metadáta, ktoré určujú kontext autorizácie a musia byť chránené proti manipulácii. V návrhu je dôležité definovať, ktoré polia sa:

- považujú za nedôveryhodné a slúžia iba ako vstup do politiky,
- musia byť kryptograficky viazané (napr. cez HMAC alebo AEAD AAD),
- musia byť odvodené na serveri (napr. časová pečiatka).

Takéto rozdelenie znižuje riziko, že server začne dôverovať údajom, ktoré útočník vie ovplyvniť.

D. Škálovanie anti-replay pri viacerých zariadeniach

Anti-replay mechanizmus musí byť v praxi navrhnutý tak, aby fungoval pre viac zariadení súčasne. To znamená, že `ReplayWindow` sa typicky udržiava per identitu zariadenia (napr. per `reader_id`). Pri návrhu je potrebné rozhodnúť:

- aká je veľkosť okna (koľko hodnôt tolerujeme),
- či tolerujeme out-of-order doručenie (napr. pri výpadkoch siete),
- a ako riešime reštart zariadenia alebo stratu stavu.

Z teoretického hľadiska je vhodné, aby server odmietol nečakaný pokles sekvencie, ale zároveň poskytol administrátorovi diagnostiku (napr. zariadenie bolo resetované). Vtedy je možné zvoliť bezpečný proces obnovy (napr. re-provisioning alebo manuálne potvrdenie resetu).

E. Časové okná, oneskorenie a „freshness“

Ak sa používa časová pečiatka, treba počítať s oneskorením siete a s tým, že klient nemusí mať presný čas. Praktický prístup je, že server generuje čas pre interné správy a pri prijímaní požiadavky používa čas iba pre audit a pre detekciu anomálií (napr. nezvyčajne dlhé oneskorenie). Ak sa predsa používa klientsky čas, musí existovať tolerancia (maximálny *skew*) a fail-closed správanie mimo tolerancie.

F. Autentifikácia hardvéru cez HMAC

HMAC podpis požiadavky zabezpečuje, že iba zariadenie s tajným kľúčom vie vytvoriť platnú požiadavku. To bráni podvrhnutiu (spoofing). V prototypovej architektúre je dôležité, že zariadenie nepoužíva administrátorský token (ten je určený iba pre UI), čo znižuje riziko zneužitia v prípade kompromitácie zariadenia.

G. Anti-replay: sekvencie a ReplayWindow

1) *Prečo je replay útok nebezpečný:* Replay útok je praktický najmä tam, kde je výsledok udalosti deterministický (napr. `ALLOW`) a útočník vie zachytiť komunikáciu. Bez anti-replay by stačilo raz zachytiť platnú požiadavku a opakovať ju.

2) *Monotónne `hw_seq`:* Zariadenie používa monotónne sekvenčné číslo `hw_seq`, ktoré sa inkrementuje pri každom skene a perzistentne ukladá do NVS. V prípade reštartu zariadenia sa hodnota obnoví. Monotónnosť je dôležitá: ak by sa `hw_seq` resetoval, útočník by mohol znovu použiť staré hodnoty.

3) *Sliding window na serveri:* Server používa mechanizmus *sliding window* (`ReplayWindow`). Udržiava: (1) `maxSeen` a (2) bitset použitia hodnôt v rozsahu `[maxSeen - window, maxSeen]`.

Ak príde `seq` príliš staré, server ho odmietne ako *too old*. Ak spadá do okna, ale už bolo použité, odmietne ho ako *replay*. Tento model je bežný v protokoloch, ktoré musia tolerovať out-of-order doručenie, ale zároveň odmietnuť duplicitné správy.

H. Freshness a časová pečiatka

V embedded zariadení nie je zaručená presnosť času. Preto server generuje `ts_unix_ms` a používa ho v hlavičke `frame`. Čas slúži pre audit, pre detekciu anomálií a pre *freshness* logiky (napr. maximálne povolené oneskorenie správy).

I. Poznámka k dôvernosti komunikácie

V prototypovej implementácii je HW požiadavka autentifikovaná (integrita a pôvod), ale dôvernosť prenosu UID na úrovni HTTP nie je v základnom režime kryptograficky chránená. V praxi je vhodné použiť TLS (HTTPS) alebo posielat' už šifrovaný payload z hardvéru. V tejto práci sa dôvernosť posilňuje najmä na úrovni úložiska (neukladanie surového UID) a na úrovni interného protokolu (AEAD frame), pričom absencia povinného TLS je vedomé zjednodušenie prototypu a nie deklarovaná bezpečnostná vlastnosť.

X. BEZPEČNOSŤ VSTAVANÉHO ZARIADENIA A SPRACOVANIE TAJOMSTIEV

Čítačka je z hľadiska bezpečnosti nepohodlný prvok. Nie je pod takou kontrolou ako server, býva fyzicky dostupná a zároveň nemá veľa výkonu ani priestoru na zložitejšie protiopatrenia. Pri návrhu preto nepomáha tváriť sa, že ide o plnohodnotne dôveryhodný uzol. Užitočnejšie je počítať s tým, že zariadenie môže byť slabším článkom a že server musí jeho správy vždy znovu overiť.

Model hrozieb preto zahŕňa situácie, v ktorých útočník získa:

- krátkodobý fyzický prístup (napr. otvorenie krytu, pripojenie k rozhraniam),
- možnosť resetovať napájanie alebo opakovane reštartovať zariadenie,
- možnosť odoberať firmware z pamäte (v závislosti od ochrany),
- možnosť ovplyvniť prostredie (Wi-Fi, rušenie, pokusy o MITM).

V praxi to znamená, že zariadenie sa nemá považovať za „dôveryhodné“ v rovnakom zmysle ako server. Zariadenie je skôr klient, ktorého správy sa musia overiť a limitovať.

A. Ukladanie tajomstiev na zariadení

Najväčším problémom pri jednoduchých schémach je dlhodobé uložené tajomstvo (napr. PSK pre HMAC). Ak útočník tajomstvo získa, vie zariadenie plnohodnotne emulovať. Preto sa v teórii odporúča:

- používať per-device tajomstvá (každé zariadenie má vlastný kľúč),
- oddeliť tajomstvá podľa účelu (autentifikácia, šifrovanie, audit),
- a uvažovať o hardvérových mechanizmoch (secure element, flash encryption).

V prostredí ESP32 existujú mechanizmy ako *Secure Boot* a *Flash Encryption*, ktoré dokážu sťažiť extrakciu firmware a tajomstiev. Pre prototyp nie je cieľom implementovať kompletný hardening, ale v teoretickej rovine je dôležité tieto možnosti poznať a zvážiť ich v produkčnom návrhu.

B. Monotónne počítadlá a trvácnosť stavu

Anti-replay mechanizmy často pracujú so sekvenčným číslom alebo časom. Aby mali význam, musia byť trvácne aj po reštarte. Z toho vyplývajú dve praktické otázky:

- Ako ukladať stav tak, aby sa pri výpadku napájania nestratil?
- Ako zabrániť rollbacku stavu (útočník obnoví starú hodnotu)?

Jednoduché perzistentné ukladanie (napr. NVS) rieši prvú otázku. Druhá otázka je zložitejšia a v produkcii sa rieši napríklad cez secure storage, monotónne čítače v hardvéri alebo serverové pravidlá, ktoré odmietnu nečakaný pokles sekvencie.

C. Bezpečnosť firmware a aktualizácie

Pri mikrokontroléri býva firmware zároveň funkciou aj bezpečnostnou hranicou. Keď sa v ňom objaví chyba, nestačí povedať, že ju „niekedy opravíme“; bez rozumného mechanizmu aktualizácie zostane slabé miesto v zariadení prakticky po celý jeho životný cyklus. Na druhej strane, ak sa aktualizácie navrhnu zle, stanú sa najkratšou cestou k prevzatiu zariadenia.

Preto sa v odbornej literatúre opakuje niekoľko rovnakých zásad: nový firmware má byť podpísaný, zariadenie má vedieť odmietnuť staršiu zraniteľnú verziu a vývojové rozhrania nemajú zostať otvorené v nasadení. V tejto práci nie je implementovaný plnohodnotný secure boot ani OTA aktualizácia; bolo by nepresné tvrdiť opak. Z pohľadu témy je však dôležité pomenovať, že práve firmware rozhoduje o tom, či sa zachová správne počítadlo `hw_seq`, či sa tajomstvo použije iba na podpis a či zariadenie neposiela na sieť viac údajov, než má.

D. Fyzické útoky a praktický hardening

Keď je čítačka pri dverách fyzicky dostupná, model hrozieb sa mení. Útočník už nemusí útočiť cez sieť; môže skúšať debug port, napájanie, vnútornú zbernicu alebo jednoducho vymeniť modul za vlastný. V laboratórnom prototypu sa proti takémuto súperovi nedá dosiahnuť rovnaká odolnosť ako v komerčnom kontroléri so secure elementom a chráneným kryptom. Dáva preto zmysel hovoriť o primeranom hardeningu, nie o úplnej fyzickej odolnosti.

Prakticky to znamená aspoň tri veci. Po prvé, vypnúť alebo obmedziť rozhrania, ktoré boli užitočné pri vývoji, ale v prevádzke by iba rozširovali útočnú plochu. Po druhé, premýšľať nad tým, čo sa stane po kompromitácii jednej čítačky; v dobre navrhnutom systéme to nesmie automaticky znamenať kompromitáciu celej serverovej časti. A po tretie, pri interpretácii výsledkov práce otvorene priznať, že fyzická ochrana zariadenia tu nie je hlavným predmetom riešenia.

E. Bezpečnosť Wi-Fi konfigurácie a lokálnej siete

Wi-Fi býva v prototypoch často najpraktickejšia voľba, ale zároveň prináša typický problém: zariadenie sa ocitne v sieti, kde nie je samo. Vedľa neho môžu byť telefóny, notebooky aj iné IoT prvky, ktoré nemusia byť dôveryhodné. Práve preto nestačí pozerieť sa iba na obsah správy; dôležité je aj to, z akej siete zariadenie komunikuje a s kým vôbec smie hovoriť.

Pre tento typ riešenia je rozumné oddeliť zariadenia aspoň do samostatnej siete alebo VLAN, zúžiť komunikáciu na nevyhnutné endpointy a nenechať otvorené všeobecné služby, ktoré prototyp nepotrebuje. Silné Wi-Fi zabezpečenie a monitorovanie neúspešných spojení sú už skôr prevádzkové disciplíny, ale v praxi často rozhodnú o tom, či incident zostane lokálny, alebo sa rozšíri ďalej. Sieťová segmentácia sama osebe kryptografiu nenahrádza; iba kupuje čas a znižuje dosah problému.

F. Bezpečnosť logovania na zariadení

Pri embedded zariadení je lákavé logovať všetko, najmä počas ladenia. Presne tu však vzniká bežná chyba: do konzoly sa dostanú informácie, ktoré tam po nasadení nemajú čo robiť. V tomto prototypu majú logy slúžiť hlavne na diagnostiku čítačky a komunikácie, nie na prenos citlivých údajov. Záznamy preto nemajú obsahovať tajomstvá, celé podpisy ani zbytočne detailné vnútorné stavy.

Rovnako dôležité je, aby sa zo servisného logovania nestal trvalý režim prevádzky. To, čo pomáha pri vývoji, môže po nasadení fungovať ako jednoduchý únikový kanál. Aj pri tak malej súčasti systému sa teda ukazuje rovnaký princíp ako inde: diagnostika je potrebná, ale musí byť podriadená bezpečnostnému cieľu, nie naopak.

XI. BEZPEČNOSŤ TRANSPORTNEJ VRSTVY A POUŽITIE TLS

Pri komunikácii medzi čítačkou a serverom sa ľahko zmiešajú dve odlišné vrstvy ochrany. Jedna otázka znie, či je dôveryhodná samotná požiadavka — teda či server vie, kto ju vytvoril a či ju nikto po ceste neupravil. Druhá otázka sa týka celého prenosu po sieti: či niekto nevie čítať alebo meniť komunikáciu na transportnej vrstve. HMAC a AEAD rieši prvý problém na úrovni správy; TLS rieši druhý na úrovni spojenia. Obe vrstvy sú užitočné, ale ich úloha nie je zameniteľná.[14], [15]

A. Prečo nestačí iba TLS

Samotné TLS neodpovie na všetko. Ak je zariadenie kompromitované, vie stále vytvárať úplne korektné HTTPS požiadavky. Ak je chyba v serverovej autorizácii, TLS ju nezakryje. A ak je v architektúre prítomná proxy alebo reverzný front-end, transportná ochrana sa môže ukončiť ešte pred aplikáciou, ktorá rozhoduje o prístupe.

Preto dáva zmysel ponechať si aj aplikačnú autentifikáciu. V tomto prototypu túto úlohu plní HMAC nad telom požiadavky a väzba na `hw_seq`. Aj keby sa transportná vrstva neskôr zmenila, server stále vie overiť, že prijatá udalosť zodpovedá tomu, čo odoslalo konkrétne zariadenie.

B. mTLS vs. symetrické tajomstvá

Pri autentifikácii zariadení sa v praxi stretávajú dve cesty. Prvá je postavená na symetrickom tajomstve, z ktorého sa počíta HMAC; druhá používa certifikáty a mutual TLS. mTLS sa lepšie hodí do väčších infraštruktúr, kde treba riešiť životný cyklus identity zariadenia, revokáciu a správu certifikátov. Má však aj cenu: PKI, distribúcia dôvery a komplikovanejšie nasadenie na strane klienta.

Pre tento prototyp bol zvolený jednoduchší model so zdieľaným tajomstvom. Nie preto, že by mTLS nebolo bezpečné, ale preto, že cieľom práce nebolo stavať certifikačnú infraštruktúru. HMAC stačí na demonštráciu autenticity zariadenia, integrity požiadavky a ochrany proti replay na aplikačnej vrstve.

C. Validácia certifikátu a certificate pinning

Ak sa TLS použije, najslabším miestom býva paradoxne klient. V embedded praxi sa stále objavujú implementácie, ktoré certifikát servera neoverujú dôsledne, alebo validáciu vypnú úplne, lebo je „jednoduchšie rozbehať spojenie“. V takom prípade ostane z TLS iba šifrovaný tunel bez skutočnej identity servera.

Pre stabilné prostredia prichádza do úvahy aj *certificate pinning*, teda uloženie očakávaného verejného kľúča alebo fingerprintu. Tým sa znižuje závislosť od všeobecného trust store, ale zároveň sa komplikuje rotácia certifikátu. Je to typický príklad mechanizmu, ktorý vyzerá čisto bezpečnostne, no v praxi sa rýchlo zmení na prevádzkový problém, ak sa zle naplňuje.[15]

D. Praktické nasadenie TLS v IoT

V teórii sa TLS často spomína ako samozrejímavá voľba. V malom IoT zariadení však za tým nasledujú konkrétne otázky: kde bude uložený CA certifikát, ako sa vyrieši jeho obmena, čo sa stane po expirácii a kto zabezpečí aktualizáciu zariadenia bez výpadku. Práve tu sa ukazuje rozdiel medzi „bezpečným mechanizmom“ a „udržateľným nasadením“.

Z tohto dôvodu ostáva v prototype základná komunikácia hardvéru riešená ako HTTP + HMAC. Neznamená to, že TLS nie je dôležité; znamená to iba to, že ťažisko práce je inde. Produkčné riešenie by malo pridať TLS alebo mTLS aj kvôli dôvernosti prenášaného identifikátora.

E. Kombinácia vrstiev: kedy dáva zmysel HMAC aj pri TLS

Otázka „prečo HMAC, keď už mám TLS?“ sa objavuje prirodzene. Odpoveď závisí od toho, čo presne chceme overiť. TLS chráni spojenie medzi dvoma bodmi. HMAC v tomto návrhu viaže konkrétnu požiadavku na zariadenie a na jej obsah. To je užitočné napríklad vtedy, keď sa transportná vrstva v budúcnosti presunie na proxy, load balancer alebo iný medzičlánok.

Inými slovami: TLS by vylepšilo dôvernosť a transportnú integritu, no HMAC zostáva nositeľom aplikačnej identity zariadenia. Tieto vrstvy sa preto nevylučujú; každá rieši inú časť problému.

XII. BEZPEČNOSŤ ÚLOŽISKA, EXPORTOV A ŽIVOTNÝ CYKLUS DÁT

Úložisko prístupového systému má dve hlavné úlohy: udržiava konfiguráciu (karty, roly, väzby) a uchováva audit udalostí. Z toho vyplýva, že únik alebo manipulácia s úložiskom má vysoký dopad.

A. Minimizácia dát a pseudonymizácia

Princíp minimalizácie hovorí, že systém má ukladať iba to, čo potrebuje. Ukladanie HMAC(card_id) namiesto surového UID je príkladom pseudonymizácie: záznamy nie sú priamo čitateľné bez tajomstva (pepper). To znižuje dopad úniku databázy a zároveň umožňuje systém používať (vyhl'adávanie karty podľa UID prebehne tak, že sa UID prepočíta na hash).

B. Export/import a integritné kontroly

Pri administrácii sa často objaví potreba exportovať databázu alebo migrovať konfiguráciu. Export/import je však rizikový: môže obísť bežné API kontroly a v prípade neovereného importu viesť k injekcii neplatných pravidiel. Z teoretického hľadiska je preto vhodné:

- vykonať integrity check databázy po importe,
- overiť auditnú reťaz, ak je súčasťou exportu,
- a auditovať samotné administrátorské operácie export/import.

C. Prístupové práva k databáze a zálohovanie

Databáza je často obyčajný súbor (pri SQLite). Z bezpečnostného hľadiska to znamená, že prístupové práva na úrovni operačného systému sú kritické: proces servera má mať práva iba na čítanie/zápis, ktoré potrebuje, a administrátorské exporty majú byť auditované. Pri zálohovaní treba myslieť na to, že záloha obsahuje rovnaké citlivé dáta ako primárna databáza.

V produkcii je vhodné uvažovať aj o šifrovaní úložiska alebo o databáze s podporou šifrovania. Šifrovanie v pokoji (at-rest) však nenahrádza auditnú integritu: útočník s právami procesu môže stále dáta meniť, preto je tamper-evident logovanie doplnkovým mechanizmom.

D. Retencia a ochrana súkromia v audite

Audit záznamy sú z pohľadu bezpečnosti užitočné, ale môžu obsahovať citlivé informácie o pohybe používateľov. V produkcii je preto potrebné definovať:

- obdobie uchovávanía (retenciu),
- prístupové práva k auditu,
- a prípadné anonymizačné alebo agregované reporty.

Tieto aspekty sú najmä procesné, ale priamo súvisia s dôveryhodnosťou systému a s právnymi požiadavkami.

XIII. OCHRANA SÚKROMIA IDENTIFIKÁTOROV A BEZPEČNOSŤ ÚLOŽISKA

A. Prečo neukladať surový UID

UID je identifikátor, ktorý môže byť v čase stabilný a unikátny. Pri úniku databázy by bolo možné:

- získať zoznam platných identifikátorov,
- korelovať pohyb používateľov podľa auditných záznamov,
- v niektorých prípadoch použiť UID na emuláciu/klonovanie.

Preto prototyp ukladá iba odvodenú hodnotu (HMAC hash).

B. $HMAC(card_id)$ s pepper

Systém ukladá do databázy hodnotu:

$$card_hash = HMAC(pepper, card_id) \quad (1)$$

kde `pepper` je tajomstvo uložené mimo databázy. Ak útočník získa SQLite súbor, bez `pepper` nevie efektívne zistiť pôvodné `card_id`. Tento prístup je analogický s odporúčaním „oddeliť tajomstvo od databázy“.

C. Pepper vs. salt a odolnosť voči slovníkovým útokom

Pri ochrane identifikátorov sa často spomína *salt*. Salt je náhodná hodnota, ktorá sa ukladá spolu s hashom a bráni tomu, aby rovnaký vstup mal rovnaký hash. V prístupových systémoch však často chceme, aby systém vedel vyhľadávať kartu podľa UID (t. j. aby rovnaký UID viedol na rovnaký `card_hash`). Preto sa v tejto práci používa pepper — tajomstvo, ktoré sa do databázy neukladá.

Pepper chráni proti tomu, aby útočník po úniku databázy spustil efektívny slovníkový útok nad priestorom UID. Pri niektorých typoch kariet môže byť priestor UID relatívne malý alebo štruktúrovaný; práve preto je dôležité, aby útočník nemal prístup k tajomstvu, ktoré hashovanie „posilňuje“. Tento prístup je analogický k bezpečnému ukladaniu hesiel, s tým rozdielom, že tu nejde o heslo používateľa, ale o identifikátor.

D. Minimalizácia údajov v audite a súkromnostné riziká

Aj keď sa surové UID do databázy neukladá, audit môže stále obsahovať informácie, ktoré umožňujú *korelovať* správanie používateľov (čas, miesto, frekvencia prístupov). Z toho vyplýva, že prístup k auditu má byť obmedzený a v produkcii je vhodné zaviesť role-based prístup k auditným záznamom, pravidlá retencie a v reportoch preferovať agregované údaje pred detailnými trasami.

E. Poznámka k právnym aspektom (GDPR)

V európskom prostredí sa pri spracovaní identifikátorov a auditných záznamov často uplatňuje GDPR. Aj keď prototyp nie je produkčný systém, teoreticky je dôležité, aby návrh rešpektoval princípy minimalizácie, účelového viazania a primeranej doby uchovávanía. Hashovanie UID s pepper znižuje riziko pri úniku databázy, ale auditné záznamy stále môžu predstavovať osobné údaje v závislosti od kontextu organizácie. Preto je vhodné v praxi doplniť interné pravidlá, kto má prístup k auditu a na aký účel.

F. Integrita databázy a hranice SQLite

SQLite poskytuje transakcie, obmedzenia a *integrity check*, ale nepodpisuje obsah databázy a sama osebe nerieši ani distribuovanú dostupnosť či správu prístupov vo väčšej infraštruktúre. Preto pri lokálnej manipulácii súboru nie je možné zaručiť integritu bez dodatočných mechanizmov. V prototypu je táto slabina čiastočne kompenzovaná tamper-evident audit logom, pričom v práci sa otvorene uvádza, že ide o lokálne úložisko vhodné pre experimentálnu a validačnú fázu.

XIV. TAMPER-EVIDENT AUDIT LOG

Audit v prístupovom systéme nemá slúžiť iba na to, aby si administrátor vedel späťne pozrieť, kto prišiel ku dverám. Má byť aj zdrojom dôkazu po incidente. To mení požiadavky na jeho návrh. Nestačí, že sa udalosti zapisujú; dôležité je, aby sa dalo neskôr rozpoznať, či niekto históriu nemenil alebo neorezal. Výskum v oblasti bezpečného logovania preto zdôrazňuje detekciu manipulácie a, pri silnejších schémach, aj *forward integrity*[9], [13].

A. Základné typy útokov na logy

- Modifikácia záznamu: zmena výsledku ALLOW/DENY alebo času.
- Vloženie záznamu: pridanie falošnej udalosti.
- Vymazanie (truncation): odstránenie časti logu, typicky konca.
- Reorder: zmena poradia udalostí.

B. Hash chain s HMAC

Prototyp používa jednoduchú, ale účinnú konštrukciu: každý záznam obsahuje HMAC hash predošlého záznamu. Zjednodušene:

$$H_i = \text{HMAC}(K_{\text{audit}}, \text{encode}(\text{record}_i) \parallel H_{i-1}) \quad (2)$$

kde K_{audit} je tajný auditný kľúč. Vďaka tomu zmena ktoréhokoľvek záznamu spôsobí nekonzistenciu v následných hodnotách.

C. Audit anchor a ochrana proti truncation

Hash chain sama o sebe nezabráni útoku, pri ktorom útočník odstráni posledných n záznamov a ponechá konzistentný prefix. Preto prototyp používa audit anchor: periodicky sa ukladá hodnota H_i a index i do samostatnej štruktúry. Pri verifikácii sa kontroluje, že audit log obsahuje všetky záznamy minimálne po posledný anchor.

D. Forward integrity a správa auditného kľúča

Výskum bezpečného logovania používa pojem *forward integrity*: aj keď útočník v čase t získa aktuálny kľúč, nemal by byť schopný spätne upraviť staršie záznamy bez detekcie. Jednoduchý hash chain s fixným kľúčom poskytuje detekciu modifikácie, ale pri kompromitácii kľúča útočník môže vytvoriť alternatívnu históriu od miesta kompromitácie. Silnejšie konštrukcie preto používajú evolúciu kľúča (kľúč sa po každom zázname alebo po epochách aktualizuje jednosmerným spôsobom).[9]

V prototypovej práci je cieľom demonštrovať tamper-evident vlastnosti a detekciu manipulačných útokov. Pre produkčný systém by bolo vhodné zvážiť aj forward-integrity varianty a pravidlá rotácie auditného kľúča.

E. Uloženie anchorov a dôveryhodný referenčný bod

Anchor zvyšuje odolnosť proti truncation, ale vytvára otázku: *kde anchor uložiť tak, aby ho útočník nevedel jednoducho prepisovať?* Teoreticky existujú možnosti: uložiť anchor mimo hlavnej databázy, replikovať anchor do externého systému alebo periodicky exportovať anchor ako „checkpoint“ mimo dosahu bežného procesu.

F. Limity a alternatívy: Merkle stromy a verifikovateľné výňatky

Hash chain je jednoduchý a efektívny, ale verifikácia typicky vyžaduje prejsť celý log (alebo aspoň od anchoru). Pre veľké objemy dát sa používajú aj Merkle stromy, ktoré umožňujú verifikovať vybrané výňatky bez nutnosti spracovať celé dáta. Tieto prístupy sú zložitejšie na implementáciu, ale môžu byť vhodné, ak audit rastie do veľkých rozmerov alebo ak sa vyžaduje časté overovanie.

G. Verifikácia a nástroj *audit_verify*

Verifikácia prebieha offline alebo cez admin panel:

- načítanie záznamov a rekonštrukcia reťaze,
- kontrola konzistencie H_i ,
- kontrola anchor proti truncation,
- reportovanie prvého indexu, kde reťaz zlyhá.

XV. ODPORÚČANIA A ŠTANDARDY RELEVANTNÉ PRE NÁVRH

Pri návrhu bezpečnostných mechanizmov je dôležité opierať sa o etablované odporúčania a štandardy. V tejto práci sú relevantné najmä odporúčania pre manažment kľúčov (NIST SP 800-57), [6] odporúčania pre zabezpečenie RFID systémov (NIST SP 800-98), [7] odporúčania pre digitálnu identitu a autentifikátory (NIST SP 800-63B-4), [12] zásady správy bezpečnostných logov (NIST SP 800-92) [13] a štandardizované kryptografické primitíva HMAC, HKDF a ChaCha20-Poly1305. [5], [10], [4], [1], [11]

Cieľom nie je „odcitovať“ štandardy, ale použiť ich ako rámec: oddelenie kľúčov podľa účelu, rotácia kľúčov, dôsledná validácia vstupov a minimalizácia dôvery v klienta.

XVI. RIADENIE RIZIKA A NÁVRH PROTIOPATRENÍ

Pri hodnotení navrhnutého systému nie je užitočné tváriť sa, že všetky hrozby majú rovnakú váhu. V malej laboratórnej zostave je realistickejšie riešiť replay, podvrhnutie požiadavky alebo úpravu auditu než napríklad veľmi sofistikované fyzické útoky na kartu. Táto kapitola preto neponúka univerzálny katalóg rizík, ale skôr vysvetľuje, prečo boli vybrané práve tie protiopatrenia, ktoré dávajú zmysel pre tento prototyp.

A. Pravdepodobnosť a dopad

Pri hodnotení rizika sa používa matica pravdepodobnosť $\times$ dopad. Napríklad replay útok je často pravdepodobný (stačí odpočúvanie siete) a jeho dopad môže byť vysoký (neoprávnený vstup). Naopak, niektoré fyzické útoky na RFID môžu mať nižšiu pravdepodobnosť v kontrolovanom prostredí, ale stále sa oplatí ich zvážiť.

B. Vrstvené zabezpečenie (*defense-in-depth*)

Vrstvené zabezpečenie znamená, že systém nestojí na jednom mechanizme. V návrhu sa to prejavuje tak, že HMAC chráni pôvod požiadavky, [5] anti-replay chráni pred opakovaním, AEAD chráni interné správy a integritu metadát [1], [3] a tamper-evident audit log chráni dôveryhodnosť histórie. [9]

C. Bezpečnosť ako proces

Bezpečnosť nie je jednorazová vlastnosť, ale proces: zahŕňa aktualizácie, rotáciu kľúčov, revíziu pravidiel, monitoring a reakciu na incidenty. Teoretická časť preto zdôrazňuje aj prevádzkové aspekty, nielen kryptografické primitíva.

D. Predpoklady a zvyškové riziko

Každý návrh implicitne stojí na predpokladoch. V tejto práci sa typicky predpokladá, že server je lepšie chránený než čítačka a že administrátorské rozhranie je prístupné iba autorizovaným osobám. Aj pri splnení predpokladov však ostáva zvyškové riziko (residual risk), napríklad kompromitácia jednej čítačky alebo dlhodobý DoS. Dôležité je, aby systém v takom prípade zlyhal bezpečne (fail-closed) a poskytol auditné stopy pre analýzu.

E. Komunikačné kompromisy a použiteľnosť

Prístupové systémy musia byť použiteľné: rýchla odozva, nízky počet falošných zamietnutí a jednoduchá správa oprávnení. To vytvára kompromisy. Napríklad príliš úzka anti-replay *window* môže spôsobovať zamietnutia pri nestabilnej sieti; príliš široká *window* zas znižuje bezpečnosť. Teoreticky je vhodné tieto parametre odvíjať od prostredia a mať možnosť ich meniť bez zásahu do protokolu (konfigurácia na serveri).

XVII. BEZPEČNOSŤ ADMINISTRÁTORSKÉHO ROZHRAINIA A PREVÁDZKOVÉ ASPEKTY

Okrem samotného protokolu je v prístupových systémoch kritické aj administrátorské rozhranie a prevádzkové procesy. Administrácia spravuje roly, väzby, karty a často umožňuje export alebo diagnostiku. Z toho vyplýva, že musí byť chránená silnejšie než bežné klienty.

A. Typické webové hrozby: XSS, CSRF a spracovanie vstupu

Administrátorské rozhranie je webová aplikácia a dedí typické webové hrozby. Aj keď ide o interné UI, zraniteľnosti ako XSS (Cross-Site Scripting) alebo CSRF (Cross-Site Request Forgery) môžu viesť k tomu, že útočník vykoná administrátorské operácie bez vedomia používateľa. Z teoretického hľadiska je preto vhodné:

- sanitizovať a escapovať výstupy v UI,
- používať CSRF tokeny pri stavových operáciách,
- validovať vstupy aj v administrátorských endpointoch (nie iba na HW endpointoch).

Pri vývoji je bežné zobrazovať logy a audit v reálnom čase. Práve tu sa často objavujú XSS riziká, ak sa zobrazujú nesanitizované reťazce pochádzajúce z vonkajších vstupov.

B. Autentifikácia administrátora a správa relácií

Administrátorské rozhranie by malo používať silnú autentifikáciu (ideálne viacfaktorovú), obmedzenú platnosť tokenov a bezpečné ukladanie relácií. Pri citlivých operáciách je vhodné vyžadovať opätovné overenie alebo rotáciu relácie.

C. Autorizácia operácií a audit administrácie

Operácie typu *pridať kartu*, *zmeniť rolu* alebo *zmeniť väzbu čítačka→dvere* majú priamy dopad na bezpečnosť. Preto je vhodné mať oddelené administrátorské roly a logovať administrátorské zmeny do auditnej stopy.

D. Bezpečnostná konfigurácia a minimalizácia „debug“ rozhraní

Debug endpointy a vývojové nastavenia sú častým zdrojom incidentov. Z teoretického hľadiska platí, že musia byť v produkcii vypnuté alebo striktné obmedzené. OWASP API Security Top 10 upozorňuje najmä na *Security Misconfiguration*. [8]

E. Monitoring a detekcia anomálií

Monitorovanie je doplnkom ku kryptografii: nárast *HMAC fail*, *replay* udalostí alebo zlyhaní verifikácie auditu môže indikovať útok alebo chybu konfigurácie. Dobre navrhnutý systém umožní nielen odmietnuť podozrivú požiadavku, ale aj poskytnúť podklady pre analýzu a zlepšenie politiky.

XVIII. SÚVISIACE PRÍSTUPY A TYPICKÉ ARCHITEKTÚRY SYSTÉMOV KONTROLY PRÍSTUPU

Nie všetky prístupové systémy vyzerajú rovnako a nebolo by správne predstavovať navrhnutý prototyp ako jediný rozumný model. Táto kapitola preto slúži skôr na zasadenie riešenia do kontextu. Ukazuje, kde sa náš prototyp podobá bežným architektúram a kde je naopak zjednodušený práve preto, že ide o akademický a experimentálny systém.

A. Centralizované vs. distribuované rozhodovanie

Pri porovnávaní architektúr je najpraktickejšie položiť si jednoduchú otázku: kde sa prijíma rozhodnutie o otvorení dverí? V centralizovanom modeli sa udalosť pošle na server, ktorý pozná pravidlá, stav systému aj auditný kontext. To zjednodušuje správu oprávnení a revokácií, ale systém sa stáva citlivejší na výpadok siete alebo servera.

Distribuované riešenie posúva časť logiky bližšie k dverám. Výhodou je vyššia autonómia pri výpadku konektivity. Cena za to je menej pohodlná správa politiky, viac lokálnych tajomstiev a väčšia dôležitosť fyzickej bezpečnosti edge zariadenia. V praxi preto mnohé systémy skončia niekde medzi týmito pólmi: rozhodovanie je čiastočne lokálne, ale audit, konfigurácia a správa identít zostávajú centralizované.

B. Online vs. offline režim a bezpečnostné kompromisy

Offline režim pôsobí na prvý pohľad lákavo, pretože dvere „fungujú aj bez siete“. Lenže práve vtedy sa ukáže, čo všetko musí viesť samotné zariadenie: uchovávať lokálne kópie oprávnení, riešiť ich zastaranie a po obnovení spojenia korektne zlúčiť audit. To nie je detail, ale zásadný bezpečnostný a prevádzkový problém.

Online model je prísnejší na dostupnosť infraštruktúry, no zjednodušuje zmeny oprávnení a robí audit okamžitejší a konzistentnejší. Preto je v tejto práci zvolená práve online, centralizovaná cesta. Neznamená to, že offline režim je nesprávny; iba posúva ťažisko problému zo servera na zariadenie a na lokálnu správu dôvery.

C. Integrácia s identitnými systémami

V reálnych organizáciách býva prístupový systém iba jednou časťou širšieho identity ekosystému. Oprávnenia často nevznikajú priamo v ňom, ale prichádzajú z HR alebo z centrálného IAM riešenia. Z bezpečnostného pohľadu je potom rozhodujúce určiť, ktorý systém je „zdroj pravdy“, ako rýchlo sa prenášajú zmeny a kto nesie zodpovednosť za revokáciu práv.

Pre prototyp takáto integrácia nie je klúčová, ale je dobré ju pomenovať už v teórii. Bez tohto kontextu by sa mohlo zdať, že správa kariet a rolí je izolovaný problém, hoci v praxi je úzko previazaná s personálnymi a prevádzkovými procesmi.

D. Audit a forenzná pripravenosť v bežných riešeniach

Audit býva v marketingových materiáloch takmer samozrejmosťou. Keď sa však objaví incident, ukáže sa rozdiel medzi bežným logovaním a auditom, ktorému sa dá veriť. Na to nestačí, aby systém iba „niečo zapisoval“. Záznamy musia mať jasný pôvod, časový kontext, konzistentný formát a ochranu proti neskoršej úprave.

Práve preto má v tomto prototypy význam tamper-evident prístup. Nie je to náhrada kompletnej forenzej platformy, ale je to praktický krok od obvyčajného logovania k auditu, ktorý sa dá dodatočne verifikovať. V akademickom prototypy je to primeraná hranica: zvyšuje dôveryhodnosť výsledkov bez toho, aby bolo potrebné budovať veľkú SIEM infraštruktúru.

E. Komunikačné protokoly medzi čítačkou a kontrolérom

Nie každá čítačka komunikuje priamo po IP. V mnohých nasadeniach je medzi ňou a serverom ešte kontrolér pri dverách a samotná čítačka používa jednoduchý lokálny protokol. Historicky boli takéto rozhrania navrhnuté najmä na prenos identifikátora, nie na kryptografickú ochranu. Dobrým príkladom je Wiegand, kde problém nevzniká až na úrovni backendu, ale už na samotnom vedení medzi zariadeniami.

Modernejšie protokoly, napríklad OSDP Secure Channel, ukazujú ten istý princíp v bezpečnejšej forme: aj na krátkej lokálnej trase má zmysel riešiť autenticitu, integritu a ochranu proti replay. V tomto zmysle sa komunikácia „čítačka → kontrolér“ veľmi nelíši od komunikácie „zariadenie → server“; mení sa skôr médium a prevádzkový kontext než samotná bezpečnostná logika.

F. Topológia: inteligentná čítačka vs. kontrolér pri dverách

Rozdiel medzi inteligentnou čítačkou a jednoduchou čítačkou s externým kontrolérom je dôležitý hlavne z pohľadu miesta, kde sa nachádzajú citlivé rozhodnutia a tajomstvá. Inteligentná čítačka je architektonicky čistejšia na kabeláž a často aj jednoduchšia na prototypovanie. Zároveň však znamená, že väčšia časť dôvery sa presúva do fyzicky dostupného zariadenia.

Model s kontrolérom pri dverách presúva rozhodovanie do chránenejšej zóny, ale zase vytvára potrebu zabezpečiť ďalší komunikačný úsek. Ani jedna z možností nie je automaticky „správna“; vhodnosť závisí od prostredia, ceny a od toho, aký silný fyzický útok považujeme za realistický.

G. Cloud vs. on-premise a dôvera v infraštruktúru

Aj pri prístupových systémoch sa dnes stretávajú dva prevádzkové svety. Cloud zjednodušuje centralizovanú správu a dostupnosť z viacerých lokalít, no zároveň presúva časť dôvery mimo organizáciu. On-premise dáva väčšiu kontrolu nad auditom a databázou, ale vyžaduje disciplinovanú správu infraštruktúry, záloh a aktualizácií.

Z pohľadu tejto práce je podstatné skôr niečo iné: aby bezpečnostné mechanizmy neboli úplne závislé od konkrétneho umiestnenia servera. Message-level HMAC, vnútorný AEAD frame a tamper-evident audit majú zmysel v oboch modeloch. Rozdiel je hlavne v tom, komu a do akej miery organizácia dôveruje na úrovni prevádzky.

H. Postavenie prototypu v tomto kontexte

Navrhnutý prototyp je najbližší centralizovanému online systému. Čítačka posiela udalosť na server, server rozhoduje a zároveň vytvára auditnú stopu. Pre potreby diplomovej práce je to výhodné, lebo bezpečnostné mechanizmy zostávajú dobre pozorovateľné: dá sa ukázať podpis požiadavky, anti-replay logika, interné spracovanie frame aj verifikácia auditu.

Ak by sa riešenie posúvalo ďalej, prirodzenými smermi by boli napríklad mTLS, opatrne navrhnutý offline režim alebo silnejšia ochrana fyzického zariadenia. Základná idea by sa však nemenila: oddeliť identitu zariadenia, rozhodovanie o prístupe a dôveryhodný záznam o udalosti.

XIX. KONTROLNÝ ZOZNAM BEZPEČNÉHO NÁVRHU PRÍSTUPOVÉHO SYSTÉMU

Na konci teoretickej časti sa oplatí zhrnúť pravidlá do praktickej podoby. Nie ako normu, ktorú treba slepo odškrtať, ale ako stručný filter pre návrh. Ak by sa mal podobný systém robiť znovu alebo rozširovať do produkcie, práve tieto body by bolo rozumné skontrolovať ako prvé.

A. Autentifikácia a autorizácia

- Klient (čítačka) musí byť autentifikovaný kryptograficky; identifikátor zariadenia nesmie byť iba „číslo v požiadavke“.
- Politika prístupu má byť oddelená od mechanizmov autentifikácie (neplieť si „kto poslal správu“ s „či má vstúpiť“).
- Pre administráciu platia prísnejšie pravidlá: silná autentifikácia, oddelené roly, audit zmien.

B. Validácia vstupu a robustnosť

- Vstupy validovať syntakticky aj semanticky; mať pevné limity veľkosti.
- Uprednostniť fail-closed správanie a jednotné chybové odpovede smerom k nedôveryhodným klientom.
- Minimalizovať riziká DoS: rýchla validácia pred náročnými operáciami, rate limiting.

C. Kryptografia a manažment kľúčov

- Používať overené primitívy (HMAC, AEAD, HKDF) a vyhnúť sa vlastným konštrukciám.
- Oddeliť kľúče podľa účelu a mať plán rotácie a revokácie.
- Správne pracovať s noncomi a zabrániť ich opakovaniu pod rovnakým kľúčom.

D. Anti-replay a časové údaje

- Anti-replay riešiť explicitne: monotónne sekvencie a serverové okno.
- Minimalizovať dôveru v klientsky čas; časové údaje generovať alebo verifikovať na serveri.

E. Úložisko, súkromie a audit

- Neukladať surové identifikátory, ak to nie je nutné; preferovať pseudonymizáciu (HMAC s pepper).
- Audit považovať za dôkazný artefakt: chrániť ho proti manipulácii (hash chain, anchor).
- Definovať retenciu a prístupové práva k auditným záznamom.

F. Prevádzka a monitoring

- Monitorovať anomálie (HMAC fail, replay, zmeny konfigurácie, zlyhanie verifikácie auditu).
- Mať procesy pre incident: blokovanie zariadenia, rotácia kľúčov, analýza auditu.

Takýto kontrolný zoznam pomáha previesť teóriu do konzistentného návrhu. V praxi sa jednotlivé body prispôbujú prostrediu a dostupným zdrojom, no princíp vrstveného zabezpečenia ostáva rovnaký.

XX. ZHRNUTIE TEORETICKÝCH VÝCHODÍSK

Teoretická časť mala pripraviť pôdu pre praktický prototyp, nie vytvoriť dojem „učebnicovo ideálneho“ systému. Preto boli popísané nielen ciele a odporúčané mechanizmy, ale aj hranice zvoleného riešenia. V centre pozornosti stoja tie hrozby, ktoré majú pre prototyp reálny význam: podvrhnutie požiadavky zariadenia, replay, manipulácia s internými metadátami a zásahy do auditnej stopy. Odpoveďou na ne sú HMAC, monotónne sekvencie, AEAD spracovanie interného frame a tamper-evident audit.

Zároveň sa ukázalo, že bezpečnosť prístupového systému nevzniká jedným rozhodnutím alebo jedným algoritmom. Je výsledkom viacerých menších, ale navzájom previazaných voľieb: čo sa považuje za dôveryhodné, ktoré údaje sa generujú na serveri, ako sa spravujú kľúče, čo sa zapisuje do databázy a ako sa neskôr overuje história udalostí. Práve takto pripravený teoretický rámec umožňuje, aby v ďalšej časti práce nadväzovali experimenty a analytické vyhodnotenie prirodzene, nie ako dodatočne prilepená kapitola.

XXI. PRAKTICKÁ IMPLEMENTÁCIA NAVRHNUTÉHO SYSTÉMU

V tejto časti už nevychádzam iba z teoretického modelu, ale z konkrétnej implementácie projektu `access-security`. Kódová báza je rozdelená do samostatných modulov, ktoré riešia kryptografiu, formát správ, rozhodovaciu logiku, úložisko, audit a administrátorské rozhranie. Aby neskoršie experimenty a analytické vyhodnotenie nevznikli „odtrhnuté“ od implementácie, považujem za potrebné najprv presne popísať, čo je v prototypu skutočne implementované, kde prebieha rozhodovanie a ktoré časti systému nesú bezpečnostnú zodpovednosť.

Praktickú časť preto rozdeľujem na dve vrstvy. Prvá vrstva opisuje samotnú implementáciu, teda štruktúru projektu, tok spracovania udalosti a hlavné mechanizmy použité v kóde. Druhá vrstva bude mať analytický charakter: tam sa budem venovať testom, meraniam a vyhodnoteniu toho, či implementované opatrenia naozaj zvyšujú odolnosť systému voči zvoleným hrozbám.

A. Architektúra implementácie

Implementácia je koncipovaná ako modulárny serverovo-riadený prototyp. Na strane zariadenia sa nachádza čítačka postavená na platforme ESP32 s modulom RC522. Jej úloha je úmyselne obmedzená: načíta UID karty, pripojí identifikátor čítačky a dverí, doplní monotónne počítadlo `hw_seq` a takto vytvorenú požiadavku autentifikuje pomocou HMAC-SHA256. Samotné rozhodovanie o prístupe neprebieha na zariadení, ale na serveri.

Serverová časť je implementovaná v jazyku C++ a pozostáva z HTTP rozhrania, rozhodovacej logiky, perzistentného úložiska a auditnej vrstvy. Dôležité je, že hardvérová požiadavka sa po prijatí na serveri nespracuje „skratkou“. Najprv sa overí jej HMAC podpis, potom sa z nej vytvorí interný bezpečný frame a až ten vstupuje do rovnakého validačného a rozhodovacieho reťazca, aký používa jadro systému. Vďaka tomu nie je hardvérový vstup samostatnou výnimkou, ale súčasťou jednotnej spracovateľskej cesty.

Tabuľka 6
Hlavné vrstvy implementácie a ich úloha v prototypu

Vrstva	Úloha
Hardvérová vrstva	ESP32 + RC522 načíta UID karty, vytvorí JSON požiadavku, zvýši lokálne počítadlo <code>hw_seq</code> a vypočíta HMAC podpis zasielaný v hlavičke <code>X-HW-Signature</code> .
HTTP a aplikačná vrstva	Modul <code>access_admin</code> poskytuje endpoint <code>/api/hw/uid</code> , administratívne API a webové rozhranie. Vstupy z HTTP vrstvy sú pred ďalším spracovaním validované a mapované na interné služby.
Rozhodovacie jadro	Moduly <code>access_core</code> a <code>access_decision</code> zabezpečujú parsovanie frame, kontrolu replay, verziu kľúča, dešifrovanie payloadu a finálne rozhodnutie „allow/deny“.
Úložisko a audit	Modul <code>access_storage</code> uchováva konfiguráciu čítačiek, rolí a kariet v SQLite a súčasne vedie tamper-evident auditný log s HMAC reťazením a anchor bodom.
Pozorovanie systému	Modul <code>runtime_events</code> zbiera prevádzkové udalosti, ktoré sú zobrazované v administrátorskom rozhraní ako live log.

Z hľadiska implementácie je praktické, že server nepracuje iba s jednotlivými triedami izolovane. V module `access_admin` je vytvorený objekt `AppState`, ktorý drží konfiguračný stav, inštanciu `KeyManager`, prístup do SQLite, auditný log, rozhodovací engine a kruhový buffer udalostí. Toto rozhodnutie zjednodušuje väzby medzi route handlermi, službami a rozhodovacím jadrom, pretože všetky potrebné komponenty sú zviazané na jednom mieste.

B. Modulárne členenie projektu

Pri čítaní kódu sa ukazuje, že projekt nie je organizovaný podľa typu súborov, ale podľa zodpovedností. Toto členenie je dôležité aj pre text práce, pretože neskoršia analytická časť bude testovať nie „celý program naraz“, ale konkrétne mechanizmy, ktoré sú viazané na jednotlivé moduly.

Tabuľka 7
Moduly projektu `access-security` a ich hlavná zodpovednosť

Modul	Zodpovednosť v implementácii
<code>crypto_lib</code>	Implementácia kryptografických primitív, najmä XChaCha20-Poly1305, HKDF a pomocných kryptografických funkcií.
<code>key_manager</code>	Načítanie master kľúča a deterministická derivácia podkľúčov pre AEAD, pepper identifikátorov kariet a auditný HMAC.
<code>protocol_lib</code>	Definícia hlavičky a frame formátu, serializácia a parsovanie správ, plus implementácia <code>ReplayWindow</code> .
<code>access_core</code>	Nízkoúrovňové spracovanie frame: validácia čítačky, väzby reader-door, kontroly replay, verzie kľúča, dešifrovanie a časová kontrola.
<code>access_decision</code>	Rozhodovacia logika po úspešnom dešifrovaní: interpretácia payloadu, hľadanie karty, kontrola role a vytvorenie rozhodnutia.
<code>access_storage</code>	SQLite perzistencia a auditný log s HMAC reťazou; zahŕňa aj nástroj na verifikáciu integrity auditu.
<code>runtime_events</code>	Jednoduchý interný event bus pre live log v administrátorskom rozhraní.
<code>access_admin</code>	HTTP server, REST API, import/export databázy, obsluha hardvérového endpointu a webové rozhranie.
<code>hw_layer</code>	Firmware pre ESP32/RC522, Wi-Fi komunikácia so serverom, HMAC podpis požiadavky a lokálne akustické/svetelné signalizovanie výsledku.

Takéto rozdelenie má pre prácu dve výhody. Po prvé, pri opise implementácie sa dá presne ukázať, v ktorom module sa realizuje konkrétny mechanizmus. Po druhé, v analytickej časti sa dajú testy naviazať na konkrétnu vrstvu: napríklad replay na `protocol_lib` a `access_core`, HMAC autentifikácia na `hw_layer` a `access_admin`, auditná integrita na `access_storage`.

C. Tok spracovania prístupovej udalosti

Z pohľadu celého systému je najdôležitejší tok jednej prístupovej udalosti. V implementácii má tento tok niekoľko jasne oddelených krokov:

- 1) Čítačka ESP32 načíta UID karty a prevedie ho na textový hexadecimálny tvar.
- 2) Firmware pripojí `reader_id`, `door_id` a monotónny čítač `hw_seq`, ktorý je uložený v `Preferences`, aby prežil reštart zariadenia.
- 3) Zo surového JSON tela sa vytvorí podpis HMAC-SHA256 a požiadavka sa odošle na endpoint `/api/hw/uid`.
- 4) Server v module `access_admin` overí podpis `X-HW-Signature`. Ak podpis neseďí, požiadavka je zamietnutá ešte pred spracovaním biznis logiky.
- 5) Po úspešnom overení sa požiadavka prevedie na interný payload `{"card_id": ..., "action": "open"}` a pomocou `KeyManager` sa získa AEAD kľúč podľa `reader_id` a aktuálnej verzie kľúča.
- 6) Server vytvorí interný frame: zostaví hlavičku `reader_id`, `door_id`, `ts_unix_ms`, `seq`, `key_version` a `nonce`; payload následne zašifruje algoritmom XChaCha20-Poly1305.
- 7) `DecisionEngine` odovzdá frame do `FrameHandler`, ktorý vykoná parsovanie, kontrolu väzby reader-door, kontrolu verzie kľúča, anti-replay kontrolu a dešifrovanie payloadu.
- 8) Po úspešnom dešifrovaní sa payload interpretuje ako žiadosť o otvorenie dverí. Server dopočíta HMAC identifikátora karty, nájde zodpovedajúcu rolu a overí, či má táto rola oprávnenie pre konkrétne dvere.
- 9) Výsledok sa uloží do auditnej stopy, odošle sa do `runtime_events` a v jednoduchnej JSON podobe sa vráti späť zariadeniu.

Dôležitý je najmä krok medzi prijatím HTTP požiadavky a volaním `DecisionEngine`. V tejto implementácii zariadenie neposiela hotový šifrovaný frame priamo po sieti. Namiesto toho posiela autentifikovanú požiadavku s UID a `hw_seq`; až server z nej vytvorí interný frame, ktorý ďalej spracúva rovnakou cestou ako zvyšok jadra. Z hľadiska práce je to dôležité, pretože sa tým jasne oddeľuje ochrana vstupu zariadenia (HMAC + sekvencia) od ochrany interného spracovania (AEAD + kontrola frame + audit).

D. Poznámka k implementačným rozhodnutiam a k ďalšiemu spracovaniu textu

Pri podrobnejšom čítaní kódu sa ukazuje, že nie všetky bezpečnostné mechanizmy sú koncentrované na jednom mieste. Napríklad anti-replay nie je iba vlastnosťou hardvéru, ale vzniká kombináciou `hw_seq` vo firmvéri, serverovej validácie HMAC požiadavky a následnej kontroly v `ReplayWindow`. Podobne audit nie je len databázová tabuľka, ale súčasť rozhodovacieho toku, pretože `DecisionEngine` zapisuje auditné udalosti priamo pri spracovaní výsledku.

Z tohto dôvodu bude vhodné v ďalšom kroku nerozoberať praktickú časť iba podľa súborov alebo tried, ale podľa mechanizmov: autentifikácia hardvérového zariadenia, anti-replay, interné AEAD spracovanie, rozhodovacia logika a auditná integrita. Až keď bude takto rozdelený opis implementácie hotový, bude mať analytická časť pevnú oporu na testy odolnosti, výkonnosti a detekcie manipulácie.

LITERATÚRA

- [1] Y. Nir, “RFC 8439: ChaCha20 and Poly1305 for IETF Protocols,” RFC Editor, 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8439.html>
- [2] A. Arciszewski, “draft-irtf-cfrg-xchacha-01: XChaCha: eXtended-nonce ChaCha and AEAD_XChaCha20_Poly1305,” IETF Internet-Draft, 2019. [Online]. Available: <https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-xchacha-01>
- [3] libsodium contributors, “XChaCha20-Poly1305 construction — libsodium documentation,” 2025. [Online]. Available: https://libsodium.gitbook.io/doc/secret-key_cryptography/aead/chacha20-poly1305/xchacha20-poly1305_construction
- [4] H. Krawczyk and P. Eronen, “RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” IETF, 2010. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc5869>
- [5] National Institute of Standards and Technology, “FIPS 198-1: The Keyed-Hash Message Authentication Code (HMAC),” 2008. [Online]. Available: <https://csrc.nist.gov/pubs/fips/198-1/final>
- [6] E. Barker, “NIST SP 800-57 Part 1 Rev. 5: Recommendation for Key Management — Part 1: General,” NIST, 2020. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final>
- [7] T. Karygiannis, B. Eydt, G. Barber, L. Bunn, and T. Phillips, “NIST SP 800-98: Guidelines for Securing Radio Frequency Identification (RFID) Systems,” NIST, 2007. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/98/final>
- [8] OWASP Foundation, “OWASP Top 10 API Security Risks — 2023,” 2023. [Online]. Available: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>
- [9] B. Schneier and J. Kelsey, “Secure Audit Logs to Support Computer Forensics,” *ACM Transactions on Information and System Security*, vol. 2, no. 2, pp. 159–176, 1999.
- [10] H. Krawczyk, M. Bellare, and R. Canetti, “RFC 2104: HMAC: Keyed-Hashing for Message Authentication,” RFC Editor, 1997. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2104.html>
- [11] D. McGrew, “RFC 5116: An Interface and Algorithms for Authenticated Encryption,” RFC Editor, 2008. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc5116.html>
- [12] D. Temoshok *et al.*, “NIST SP 800-63B-4: Digital Identity Guidelines — Authentication and Authenticator Management,” NIST, 2025. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/63/b/4/final>
- [13] K. Kent and M. Souppaya, “NIST SP 800-92: Guide to Computer Security Log Management,” NIST, 2006. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/92/final>
- [14] E. Rescorla, “RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3,” RFC Editor, 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8446.html>
- [15] Y. Sheffer, D. Benjamin, M. Thomson, and others, “RFC 9325: Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS),” RFC Editor, 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9325.html>
- [16] OWASP Foundation, “OWASP Application Security Verification Standard (ASVS),” [Online]. Available: <https://owasp.org/www-project-application-security-verification-standard/>
- [17] R. Sandhu, D. Ferraiolo, and D. Kuhn, “The NIST Model for Role-Based Access Control,” in *Proceedings of the 5th ACM Workshop on Role-Based Access Control*, 2000, pp. 47–63.
- [18] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, “Role-Based Access Control Models,” *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [19] National Institute of Standards and Technology, “FIPS 201-3: Personal Identity Verification (PIV) of Federal Employees and Contractors,” 2022. [Online]. Available: <https://csrc.nist.gov/pubs/fips/201-3/final>